

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ "ЛЬВІВСЬКА ПОЛІТЕХНІКА"

Д.О. Кушнір, Я.С. Парамуд

ЛАБОРАТОРНИЙ ПРАКТИКУМ
з дисципліни
“ПЕРИФЕРІЙНІ ПРИСТРОЇ,
ІНТЕРФЕЙСИ ТА ДРАЙВЕРИ”

Навчальний посібник

*Рекомендувала Науково-методична рада
Національного університету “Львівська політехніка”*

Львів
Видавництво Львівської політехніки
2026

Лабораторний практикум містить комплекс із методичних матеріалів до виконання п'яти лабораторних робіт із дисципліни “Периферійні пристрої, інтерфейси та драйвери”.

УДК 004

© Кушнір Д.О., Парамуд Я. С., 2026

© Національний університет

“Львівська політехніка”, 2026

ЗМІСТ

ЗМІСТ.....	3
ВСТУП.....	4
ТЕОРЕТИЧНІ ВІДОМОСТІ.....	5
ДО ЛАБОРАТОРНИХ РОБІТ № 1–2	5
ЛАБОРАТОРНА РОБОТА № 1	16
ЛАБОРАТОРНА РОБОТА № 2	20
ТЕОРЕТИЧНІ ВІДОМОСТІ.....	26
ДО ЛАБОРАТОРНОЇ РОБОТИ № 3.....	26
ЛАБОРАТОРНА РОБОТА № 3	32
ТЕОРЕТИЧНІ ВІДОМОСТІ.....	37
ДО ЛАБОРАТОРНИХ РОБІТ № 4–5	37
ЛАБОРАТОРНА РОБОТА № 4	44
ЛАБОРАТОРНА РОБОТА № 5	49
ДОДАТКИ	55
ДОДАТОК А. ЕЛЕКТРОННІ РЕСУРСИ КУРСУ.....	55
ДОДАТОК Б. РЕКОМЕНДОВАНА ЛІТЕРАТУРА	56

ВСТУП

Лабораторний практикум з навчальної дисципліни "Периферійні пристрої, інтерфейси та драйвери" призначений для студентів третього курсу освітньо-кваліфікаційного рівня "Бакалавр" галузі знань F2 "Інформаційні технології" спеціальності F7 "Комп'ютерна інженерія" спеціалізації "Системне Програмування". Лабораторний практикум містить методичні матеріали для п'яти лабораторних робіт. Лабораторні роботи орієнтовані на поглиблене вивчення особливостей, характеристик найбільш поширених периферійних інтерфейсів та отримання навиків розроблення для цих інтерфейсів програмних драйверів, які є складовими компонентами багатьох комп'ютерних систем. У лабораторному практикумі розглядаються послідовний асинхронний інтерфейс RS-232C, універсальна послідовна шина УППШ та синхронна магістральна шина I2C. Завершальна лабораторна робота передбачає інтеграцію вивчених інтерфейсів у складі єдиного вузла моніторингу та дослідження рівнів програмного забезпечення периферійної підсистеми. Матеріали лабораторного практикуму дозволять студентам економніше використати час на якісне виконання комплексу лабораторних робіт.

ТЕОРЕТИЧНІ ВІДОМОСТІ ДО ЛАБОРАТОРНИХ РОБІТ № 1–2

Лабораторні роботи № 1–2 призначені для поглибленого вивчення особливостей, характеристик інтерфейса RS-232C, одного із найбільш поширених периферійних інтерфейсів, та отримання навиків розроблення для цього інтерфейса програмних драйверів, які є складовими компонентами багатьох комп'ютерних систем.

Інтерфейс RS-232C за класифікацією послідовний, радіальний, дуплексний асинхронний інтерфейс. Він реалізовує стандарт передачі даних RS-232C (Reference Standard number 232 version C – стандарт обміну номер 232 версії C), який дозволяє будувати системи обміну даних між пристроями із різними швидкостями. Сучасні комп'ютерні технології широко застосовують для передачі даних послідовні інтерфейси. Інтерфейс RS-232C є одним із найстаріших та класичним представником цього класу інтерфейсів. Відповідно вивчення цього інтерфейсу є обґрунтованим.

Інтерфейс RS-232C розроблено в 1969 році низкою корпорацій США для забезпечення з'єднання комп'ютерів та різноманітних периферійних пристроїв. Аналогом інтерфейсу RS – 232C для колишніх країн економічної співдружності є спрощений варіант інтерфейсу стик C2. Інтерфейс переважно використовується для обміну даних між периферійним пристроєм та комп'ютером чи між двома комп'ютерами.

Термін “послідовний” означає, що передача даних виконується по одиночному провіднику, а біти передаються послідовно один за другим. У певний момент часу в одному напрямку передається тільки один біт даних.

Термін “радіальний” означає, що інтерфейс забезпечує взаємодію тільки двох пристроїв, один із яких є комп'ютером.

Термін “дуплексний” означає, що інтерфейс забезпечує можливість передачі даних між пристроями в обох напрямках, використовуючи для цього два окремі сигнальні провідники чи дві окремі сигнальні шини.

Термін “асинхронний” означає, що при передачі даних не використовуються ніякі синхронізуючі сигнали, і окремі символи можуть передаватись з випадковими інтервалами, як, наприклад при вводі даних з клавіатури. Інтерфейс використовує оригінальну синхронізацію, коли кожному символу, який передається через послідовне з'єднання, мусить передувати стандартний стартовий сигнал, а завершувати його передачу – стоповий сигнал. Стартовий сигнал – це умовно нульовий біт, його ще називають стартовим бітом. Його призначення – повідомити від передавального пристрою другому пристроєві, який приймає дані, про те, що наступні біти представляють собою поле даних. Поле даних, залежно від наперед запрограмованих режимів функціонування, може мати 5, 6, 7, або 8 біт. Після поля даних передається запрограмований біт контролю, а завершується обмін одного слова передачею одного, чи півтора, чи двох стопових бітів, які дають сигнал про закінчення передачі відповідного слова. У пристроєві, який приймає дані, біти розпізнаються за появою стартового сигналу по середині кожного такту, а не до моменту їх передачі. Тривалість одного такту визначається запрограмованою швидкістю обміну даними для конкретного режиму функціонування інтерфейсу. Асинхронний інтерфейс, орієнтований на передачу

окремих слів, передбачає значні непродуктивні затрати для ідентифікації кожного слова. Такі затрати можуть перевищувати 25 %.

Інтерфейс RS-232C широко використовують у персональних та промислових комп'ютерах для обміну інформацією з периферійними пристроями у послідовному дуплексному режимі обміну. У багатьох комп'ютерах інтерфейс реалізований як комунікаційний порт (COM-Port). Таких портів в одному комп'ютері може бути декілька. Інтерфейсний стандарт передбачає конструктивну сумісність пристроїв. Відповідно для кожного порта використовується стандартизований 25- чи 9-контактний з'єднувач.

Багато компаній випускають доповняльні послідовні порти для комп'ютерів, звичайно ті порти встановлюються на багатофункціональних платах чи на платі з паралельним портом. На рис. 1 показано приклад використання стандартного 9-контактного з'єднувача послідовного порта.

До послідовного порта можна підключити самі різноманітні пристрої: модеми, плотери, принтери, інші комп'ютери, пристрої зчитування штрих-кодів, схему керування пристроями чи макетну плату. Загалом у всіх пристроях, для яких потрібен низькошвидкісний, простий, двонапрямлений зв'язок з комп'ютером, використовується відповідний порт.

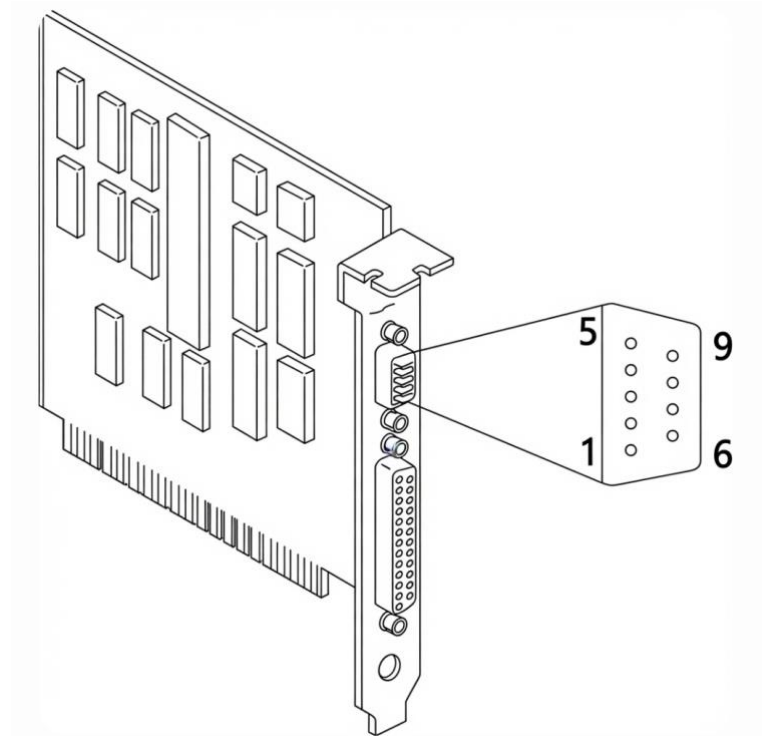


Рис. 1. Приклад використання 9-контактного з'єднувача послідовного порта

Стандарт RS-232 у загальному випадку описує 4 інтерфейсні функції:

1. Визначення керувальних сигналів, що передаються через інтерфейс.
2. Визначення формату даних користувача, що передаються через інтерфейс.

3. Передачу тактових сигналів для синхронізації потоку даних.

4. Формування електричних характеристик інтерфейсу.

Інтерфейс RS-232 є послідовним асинхронним інтерфейсом, в якому перед бітами даних передається спеціальний стартовий біт, після бітів даних йде біт паритету та 1 чи 2 стопових біти. Така сукупність бітів носить назву старт-стопного символу. Кожний старт-стопний символ, як правило, у якості бітів даних містить один інформаційний символ, наприклад, символ стандартного коду для обміну інформацією, що задається таблицею ASCII. Символи ASCII, що відображаються семибітовими кодовими словами, можуть передаватися із використанням 7-бітового поля даних. Так, наприклад, латинська буква А має код 1000001. Її передача рівнями транзисторно-транзисторної логіки (ТТЛ) зображена на часовій діаграмі (рис. 2, а). Передача рівнями сигналів RS - 232C для ліній зв'язку даних зображена на часовій діаграмі (рис. 2, б). Аналогічно формуються сигнали та відповідна часова діаграма при використанні 8-розрядної таблиці кодувань символів ASCII+.

Початок асинхронного символу завжди відмічається логічним нулем стартового сигналу (логічний "0"). Після нього йдуть 7 біт даних, потім біт паритету, та 2 стопових біти. Біт паритету встановлюється в логічні "1", або "0", наприклад, непарний паритет характеризується тим, що загальна кількість одиниць в групі з семи біт (сім даних + один біт паритету) повинно бути непарним числом. Стопові біти передаються високим рівнем (логічна "1").

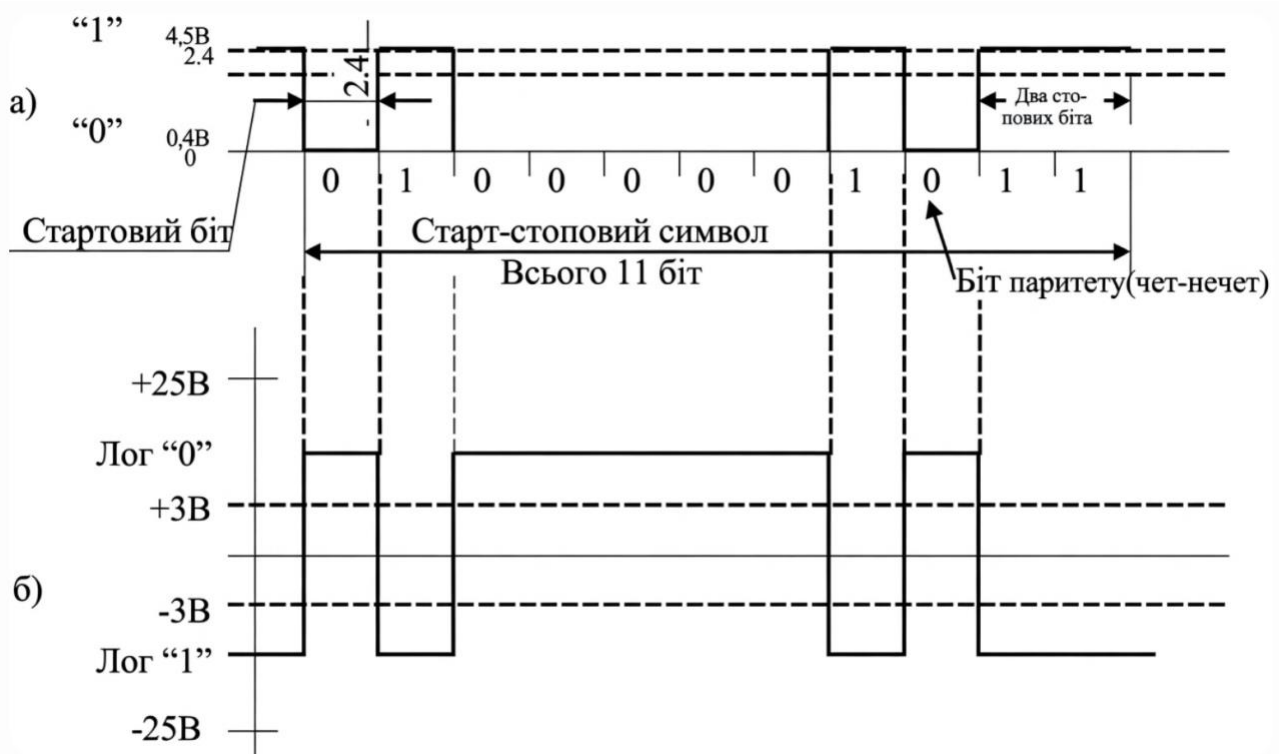


Рис. 2. Часові діаграми передачі символу для інтерфейсу RS-232C

Комп'ютер має 25-контактний (DB25P) або 9-контактний (DB9P) з'єднувачі для підключення пристрою інтерфейсу. Розташування контактів з'єднувачів зображене на

рис. 3. засвідчує, що контакти з'єднувача DB25P використовуються неповно, тому більш ефективним є використання з'єднувача DB9P. Часто у з'єднувачі DB25P використовується контакт № 1 для під'єднання захисного екрана. Використання стандартизованих з'єднувачів спрощує підключення до комп'ютера різноманітних периферійних пристроїв.

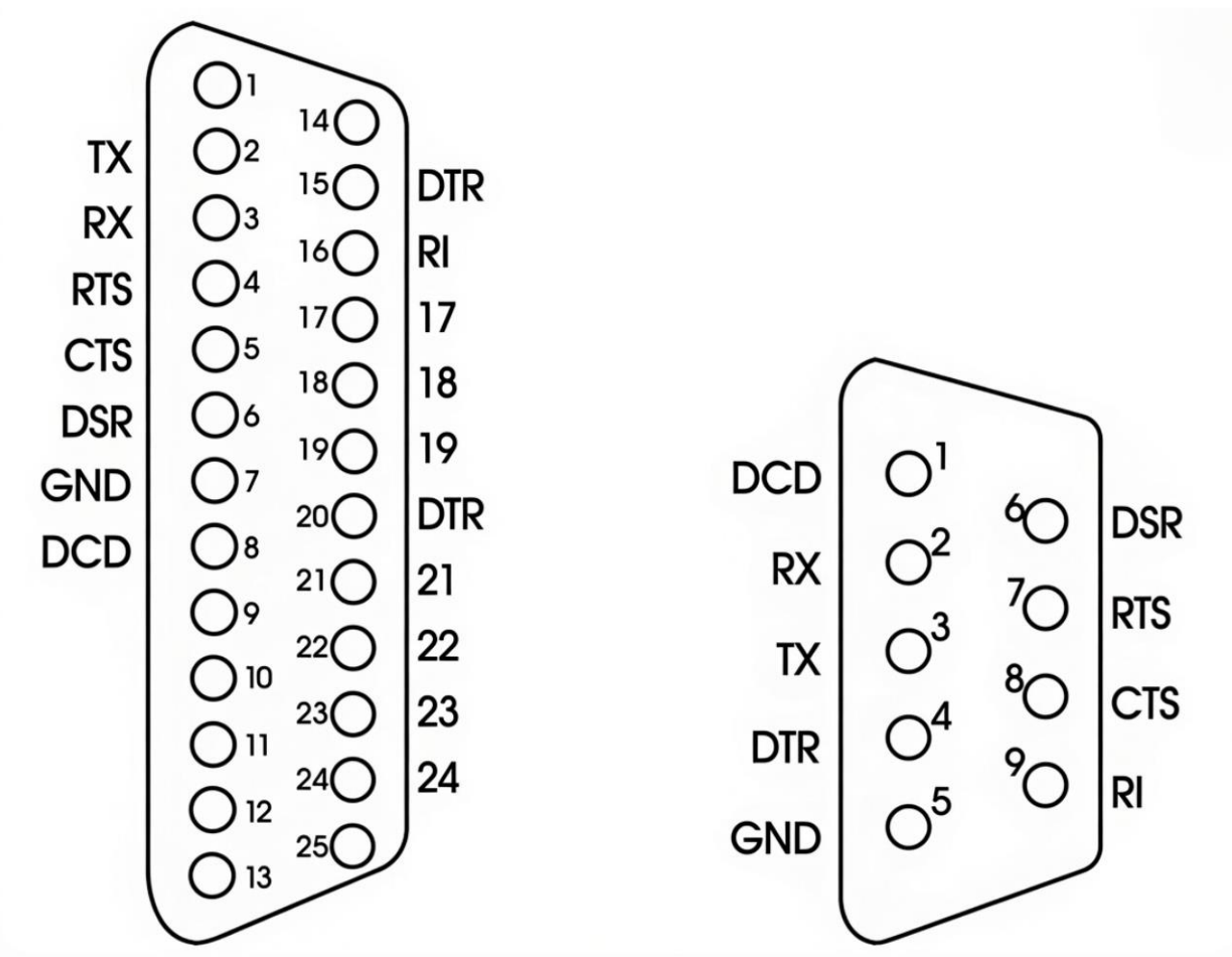


Рис. 3. Розташування контактів з'єднувачів DB25P та DB9P

Лінії зв'язку інтерфейсу можна умовно розділити на дві групи: шину даних та шину управління. Шина даних використовує дві лінії зв'язку для даних на передавання TXD та даних на приймання RXD. Цим забезпечується можливість реалізації повного дуплексного обміну даними. Шина управління має 6 ліній зв'язку, які використовуються для реалізації апаратного протоколу управління потоками даних. Призначення контактів з'єднувачів та відповідних сигналів приведене в табл. 1.

У випадку програмного протоколу управління потоками даних, коли застосовується спеціальний програмний драйвер, шина управління не використовуються тільки сигнали шини даних, що зменшує загальну кількість ліній зв'язку. Тоді застосовується три- або чотирипровідний зв'язок (для двонаправленої передачі). Схема з'єднання для чотирипровідної лінії зв'язку показана на рис. 4.

Призначення контактів з'єднувачів DB25P та DB9P

Найменування	Контакт (з'єднувач DB25P)	Контакт (з'єднувач DB9P)	Напрямок	Опис
DCD	8	1	IN	Carrie Detect (Визначення несучої)
- RXD	3	2	IN	Receive Data (Дані, що приймаються)
- TXD	2	3	OUT	Transmit Data (Дані, що передаються)
DTR	20	4	OUT	Data Terminal Ready (Готовність терміналу)
GND (SG)	7	5	-	System Ground (Сигнальна земля)
DSR	6	6	IN	Data Set Ready (Готовність даних)
RTS	4	7	OUT	Request to Send (Запит на відправку)
CTS	5	8	IN	Clear to Send (Готовність прийому)
RI	22	9	IN	Ring Indicator (Індикатор викликів)
FG	1	-	-	Захисне заземлення, екран

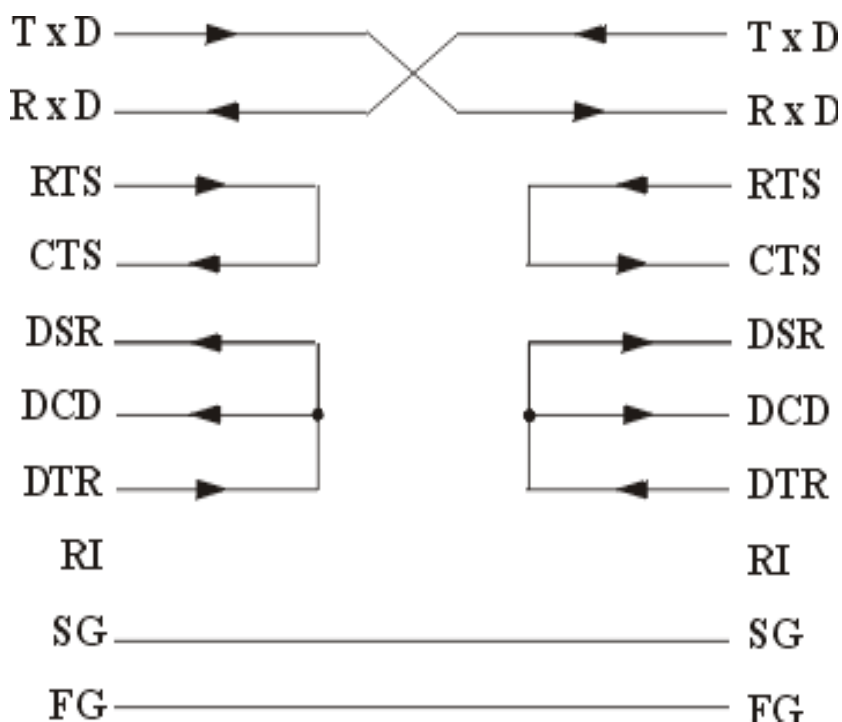


Рис. 4. Схема з'єднання 4-провідна для RS-232C

Для двопровідної лінії зв'язку у випадку передачі з комп'ютера на зовнішній пристрій використовуються сигнали SG і TxD. Всі 10 сигналів інтерфейсу задіюються лише при з'єднанні комп'ютера з модемом.

Формат слів, що передаються, показаний на рисунку 5. Рівень "1" відповідає логічній "1", рівень "2" – логічному "0". Формат програмується завчасно і має бути однаковим для передавача та приймача. Однаково мають бути запрограмованими швидкості передачі даних, які визначають тривалість одного такту. Тривалість одного такту є оберненою величиною до швидкості передавання даних. Якщо швидкість передавання даних задана в біт/с, тоді тривалість одного такту буде в долях секунди. Власне дані (5, 7 або 8 біти) супроводжуються стартовим бітом, бітом парності і одним або двома стоповими бітами. Отримавши передній фронт стартового біта, приймач переходить у режим само-синхронізації, формує тактові інтервали та очікує, коли пройде половина такту. Тоді приймач перевіряє наявність стартового сигналу. Якщо стартовий сигнал присутній, приймач встановлює, що йому від передавача поступає слово. Він вибирає з лінії зв'язку біти даних через визначені інтервали часу, на середині відповідного такта. Дуже важливо, щоб частоти синхронізуючих генераторів приймача і передавача були однаковими. Допустима розбіжність не більше 10 %, оскільки може відбутися втрата синхронізації і відповідно помилка обміну даними.

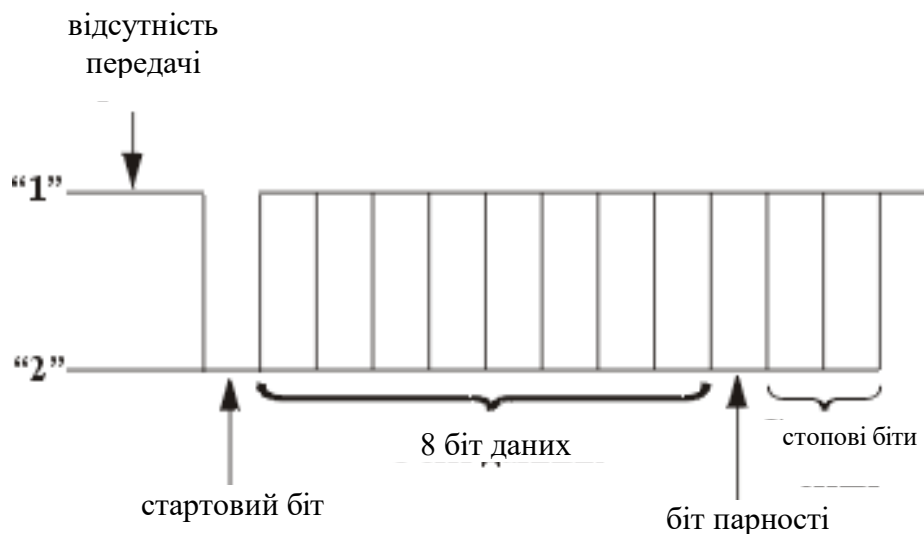


Рис. 5. Формат слова у логічних рівнях для RS-232C

Усі сигнали через лінії зв'язку RS-232C передаються спеціально вибраними рівнями, що забезпечують високу завадостійкість передачі даних (рис. 6). Відзначимо, що дані передаються в інверсному коді (логічній одиниці відповідає низький рівень, логічному нулю - високий рівень). Логічний нуль для передавача кодується величиною сигналу від +5 В до +15 В, а для приймача від +3 В до +15 В. Логічна одиниця для передавача кодується величиною сигналу від мінус 5 В до мінус 15 В, а для приймача від мінус 3 В до мінус 15 В. У

випадку використання нестабілізованих джерел живлення граничні значення сигналів можуть досягати 25 В.

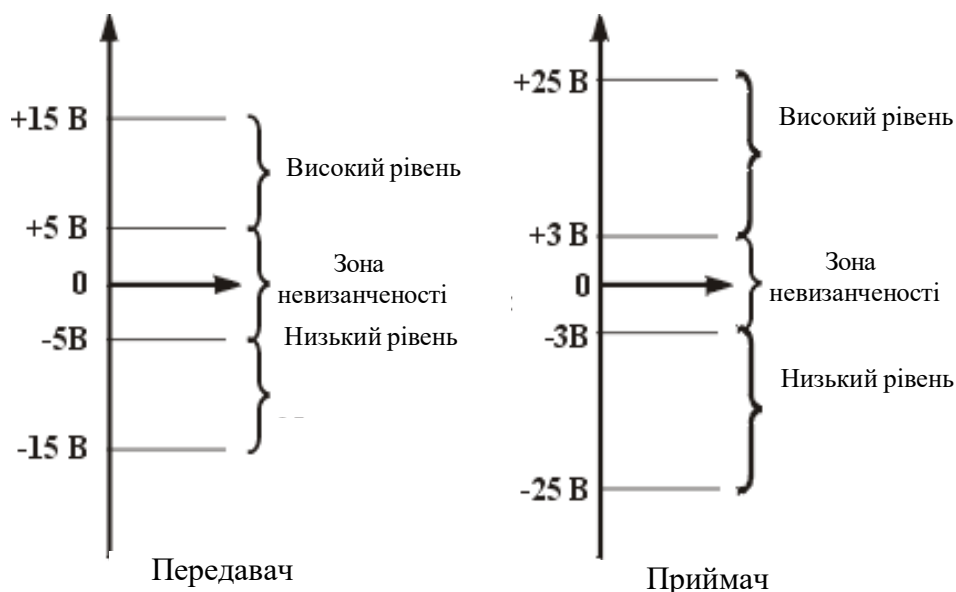


Рис. 6. Рівні сигналів RS-232C на передавальному і приймаючому кінцях лінії зв'язку

Швидкість передавання даних в асинхронному режимі може становити: 50, 75, 100, 150, 300, 600, 1200, 2400, 4800, 9600, 19200, 38400, 57600, або 115200 біт/с. Швидкість програмується при налаштуванні режимів функціонування інтерфейсу і має бути однаковою як для передавального пристрою так для приймального пристрою.

Обмін по RS-232C здійснюється за допомогою звернень по спеціально виділених для цього портах COM1 (адреси 3F8h...3FFh, переривання IRQ4), COM2 (адреси 2F8h...2FFh, переривання IRQ3), COM3 (адреси 3F8h...3EFh, переривання IRQ10), COM4 (адреси 2E8h...2EFh, переривання IRQ11). Формати звернень по цих адресах можна знайти в описах мікросхем контролерів послідовного обміну UART (Universal Asynchronous Receiver/Transmitter), наприклад, i8250, KP580BB51.

Управління потоком даних (Flow Control) інтерфейсу має наступні особливості. Воно передбачає формування повідомлень про можливість чи неможливість передавання наступного слова. Можуть використовуватися два типи протоколу управління потоком даних: апаратний чи програмний.

Апаратний протокол управління потоком даних (Hardware Flow Control) на шинах даних передавання та приймання (TXD, RXD) використовує сигнали управління "Запит на передавання" (RTS), "Готовність приймання" (CTS). Сигнал CTS забезпечує зупинку передавання даних, якщо приймач не готовий їх приймати. Передавач передає у лінію зв'язку TXD чергове слово тільки при активному стані лінії CTS. Слово, яке вже почало

передаватися, зупинити сигналом CTS неможливо, його передавання має завершитися. Це гарантує цілісність передачі. Якщо у периферійній підсистемі апаратний протокол не використовується, у передавача має забезпечуватися стан “включено” на лінії CTS перемиканням із лінією RTS. Апаратний протокол використовує лінії зв’язку шини управління, що збільшує вартість периферійної підсистеми. Однак забезпечує мінімальний час реакції передавача на стан при приймача, що підвищує загальну продуктивність інтерфейсних засобів. Оскільки вартість ліній зв’язку інтерфейсної магістралі достатньо велика, то на практиці застосування апаратного протоколу управління потоком даних постійно зменшується.

Програмний протокол управління потоком даних (Software Flow Control) використовує тільки шину даних (лінії TXD, RXD). При цьому використовуються спеціальні байт-символи XON/XOFF. Протокол функціонує наступним чином. Якщо приймач в процесі приймання даних встановить, що він не зможе правильно продовжувати приймання, він через другу лінію зв’язку шини даних посилає передавачу байт-символ XOFF (13h). Передавач, отримавши цей байт-символ, не передає наступного слова. Якщо приймач знову буде готовим до приймання даних, він посилає байт-символ XON (11h). Передавач після його отримання відновлює передавання даних. Час реакції передавача на зміну стану приймача у порівнянні із апаратним протоколом збільшується щонайменше на час передавання байт-символу XON чи XOFF плюс час реакції драйвера передавача на приймання символу. При такому протоколі управління приймач має мати буфер достатньої величини для приймання даних. Додатковим недоліком є складність забезпечення повного дуплексного режиму обміну даними. Однак мінімізація дорогих ліній зв’язку за рахунок виключення сигналів управління робить програмний протокол управління потоком даних широко вживаним.

Апаратні драйвери інтерфейсу переважно реалізуються за допомогою спеціалізованих мікросхем. Перетворення паралельного коду від ядра комп’ютера в послідовний для передачі через лінію зв’язку та зворотнє перетворення при прийманні даних виконують спеціалізовані мікросхеми UART (Universal Asynchronous Receiver Transmitter – універсальний асинхронний передавач та приймач). Вони формують та опрацьовують керуючі сигнали інтерфейсу. Апаратні драйвери інтерфейсу в IBM PC сумісних комп’ютерах переважно реалізуються на мікросхемах, сумісних на рівні регістрів з UART i8250; 8250; 8250A; 8250B; 16450; 16550. 16550A. Функціональні можливості та характеристики цих мікросхем постійно покращувалися і найбільш досконалою є остання.

Мікросхема UART i16550A як програмна модель є набором регістрів, доступ до яких відбувається за певною адресою (зміщенням адреси регістра відносно базової адреси порта) та значенням біта DLAB (біта 7 регістра налаштування параметрів каналу LCR). В адресному просторі мікросхема займає 8 суміжних адрес. Регістри мікросхеми використовуються для програмування апаратного драйвера на одну із стандартних швидкостей обміну да-

ними, кількості біт (5,6,7,8) у полі даних слова, тип контролю (парність, непарність) чи відсутність контролю, кількості біт у полі стоп слова.

Мікросхеми UART використовують рівні сигналів ТТЛ (транзисторно-транзисторна логіка) із діапазоном зміни напруги 5В. Сигнали комунікаційних портів і відповідно інтерфейсних ліній зв'язку використовують RS сигнали в діапазоні від мінус 12 В до +12 В. Для узгодження із сигналами ліній зв'язку інтерфейсу використовуються спеціалізовані мікросхеми перетворення рівнів сигналів ТТЛ у рівні сигналів RS-232C при передаванні та зворотнього перетворення при прийманні.

Амплітудно-часова діаграма UART – послідовність логічних рівнів на лінії передачі протягом формування старт-стопного кадру кожного символу. У спокої (idle) лінія перебуває в стані логічної 1. Початок символу відмічається стартовим бітом (логічний 0), далі передаються біти даних (5–8), за потреби — біт паритету, після чого — один або кілька стопових бітів (логічна 1). Між сусідніми символами може передбачатися пауза фіксованої тривалості (зокрема один такт).

Позначення формату кадру (8N1, 7E1, 8N2 тощо) визначає кількість біт даних, наявність і тип контролю парності та кількість стопових бітів. Тривалість одного такту обернено пропорційна швидкості обміну:

$$T_{\text{такт}} = \frac{1}{\text{baudrate}}$$

Час передавання повідомлення оцінюють через кількість символів, кількість тактів на символ (з урахуванням паузи між символами) і величину $T_{\text{такт}}$. На Рис. 7 зображено приклад амплітудно-часової діаграми UART.

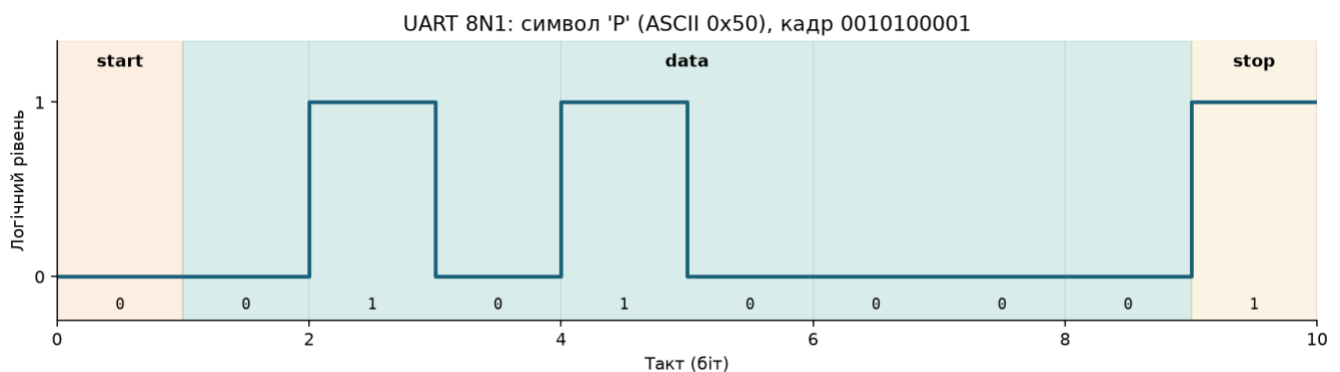


Рис. 7. Приклад амплітудно-часової діаграми UART.

Слід розрізняти логічні рівні в моделі сигналу (0 і 1) та електричні рівні RS-232C на фізичній лінії (інверсний код, діапазони напруг на передавальному та приймальному кінцях), розглянути вище.

Для послідовних високошвидкісних шин, зокрема інтерфейсу USB (УПШ), застосовують лінійне кодування, відмінне від старт-стопного кадру UART. У USB 2.0 (Full Speed / Low Speed) використовується схема NRZI (Non-Return to Zero Inverted):

- при передачі логічного 0 рівень сигналу на лінії змінюється;
- при передачі логічної 1 рівень сигналу не змінюється.

Довга послідовність одиниць не створює переходів на лінії, що ускладнює підтримання синхронізації. Тому застосовують bit stuffing: після шести послідовних одиниць у потік вставляють біт 0, який забезпечує зміну рівня (Рис 8, 9, 10).

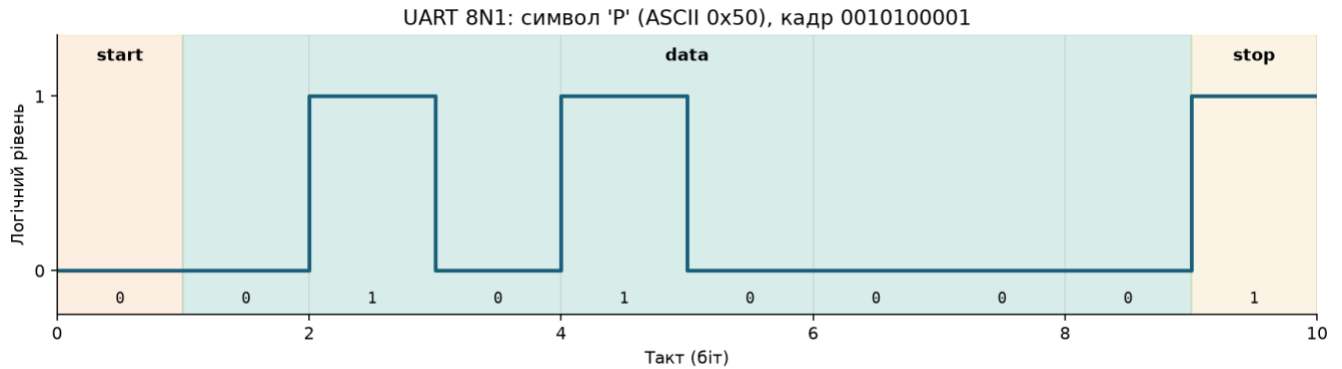


Рис. 8. Принцип кодування NRZI. методички (логічний 0 - зміна рівня, логічна 1 - рівень без зміни).



Рис. 9. Ілюстрація bit stuffing (вставка біта 0 після шести послідовних одиниць).

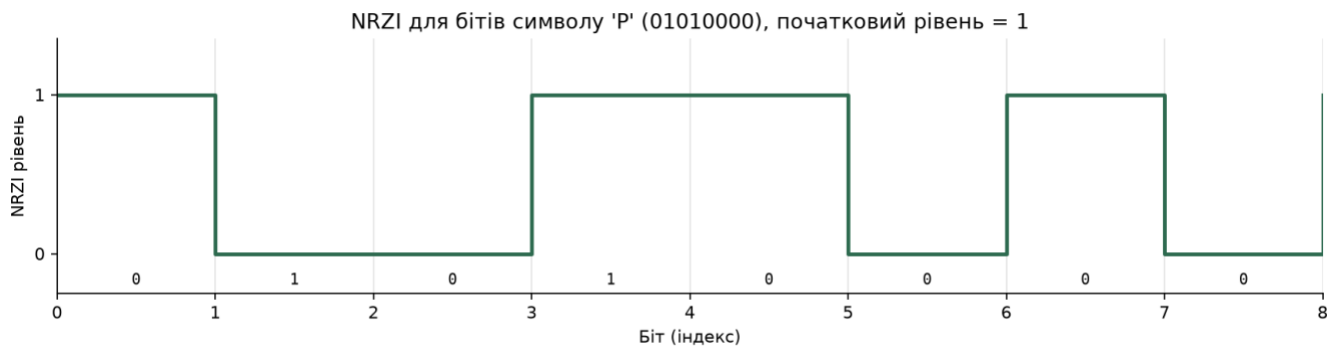


Рис. А.10. Приклад діаграми NRZI

При аналізі одного інформаційного символу спочатку розглядають його бітове представлення (наприклад, восьмибітовий код ASCII), далі — відповідний кадр UART (із start/stop/parity за обраним форматом) і окремо — кодування NRZI для бітів даних (з урахуванням bit stuffing за потреби).

Для USB 3.x SuperSpeed застосовують інші схеми кодування (зокрема 8b/10b та подальші), а не класичний NRZI USB 2.0.

ЛАБОРАТОРНА РОБОТА № 1

ДОСЛІДЖЕННЯ ПРОГРАМ ПЕРЕДАВАЧА, ПРИЙМАЧА ТА МОДЕЛІ ОБМІНУ ДАНИМИ ІНТЕРФЕЙСА RS-232C

Мета роботи: опанування студентом технології та процесу налаштування передавального та приймального портів, дослідження програм передавача і приймача пакетних даних та реалізації моделі обміну заданим повідомленням через послідовний асинхронний інтерфейс RS-232C (COM-порт / UART).

Завдання на роботу. Конкретний пакет даних та відповідне повідомлення студенту визначає викладач (переважно прізвище студента латиницею, наприклад, IVANOV). За [варіантом](#) задаються швидкість обміну даними, тип контролю, кількість стопових біт.

Технологія виконання роботи. У ході виконання цієї лабораторної роботи, необхідно просимулювати програми передавача (TX) та приймача (RX) та виконати живий обмін байтами через UART. Приклади реалізації:

Передавач: [host/uart_host.py](#). Відкриває послідовний порт, формує повідомлення (прізвище студента латиницею) і за допомогою функції `send_message()` передає його в лінію зв'язку, завершуючи пакет символом повернення каретки `\r`. За наявності ключа `--wait-ack` передавач після відправлення очікує від приймача підтвердження у вигляді рядка `ACK:...` і перевіряє коректність відповіді.

Приймач: [host/uart_device_emu.py](#). Зчитує з порту вхідний рядок до символу `\r`, виводить у журнал прийняті дані у текстовому та шістнадцятковому вигляді, а також формує й надсилає назад підтвердження у форматі `ACK:<повідомлення>\n`, чим завершує цикл обміну «запит – підтвердження».

У цій лабораторній роботі формат кадру UART є сталим – 8N1. Де 8 – це біт даних; N – без біта парності; 1 – стоп-біт.

Кроки виконання:

У прикладах використано прізвище PETRENKO і baudrate 9600; Під час виконання лабораторної потрібно підставити власне прізвище (латиницею A–Z) та параметри свого [варіанту](#).

Крок 1. Визначення вихідних даних. За таблицею варіантів і правилом повідомлення студент фіксує: номер варіанту, baudrate, формат кадру та повідомлення – власне прізвище латиницею. Це саме повідомлення має бути на стороні передавача (`--message`) і в прийнятому кадрі приймача.

Крок 2. Перевірка передавача (TX). Перед повним обміном доцільно перевірити, що програма `host/uart_host.py` коректно формує та записує кадр у порт. Для цього використовують віртуальний порт `loop://`

```
python3 -m host.uart_host --message "PETRENKO" --baud 9600 --port loop://
```


Очікуваний вивід консолі:

```
TX text: 'PETRENKO'  
Надіслано байт: 9  
TX hex: 50 45 54 52 45 4e 4b 4f 0d  
RX text (loopback): 'PETRENKO'  
Verify: OK
```

У рядку TX hex останній байт 0d – символ \r (завершення пакета).

Крок 3. Створення віртуальної пари портів (міст). Для живого шляху байтів між передавачем і приймачем на одному ПК створюють віртуальну пару COM. На Linux/macOS у першому терміналі запускають і залишають працюючим:

```
python3 -m host.uart_pty_pair
```

Типово утворюються порти /tmp/comA і /tmp/comB.

На Windows використовують com0com (наприклад, COM5 ↔ COM6).

Крок 4. Запуск приймача (RX). У другому терміналі запускають емулятор пристрою на одній половині пари:

```
python3 -m host.uart_device_emu --port /tmp/comB
```

Програма очікує вхідний кадр, що завершується символом \r.

Крок 5. Запуск передавача (TX) з очікуванням підтвердження. У третьому терміналі запускають host на другій половині пари з ключем --wait-ack:

```
python3 -m host.uart_host --message "PETRENKO" --port /tmp/comA --wait-ack
```

Очікуваний вивід передавача:

```
TX text: 'PETRENKO'  
Надіслано байт: 9  
TX hex: 50 45 54 52 45 4e 4b 4f 0d  
RX text: 'ACK:PETRENKO'  
Verify: OK
```

Очікуваний вивід приймача:

--- exchange ---

RX text: PETRENKO

RX bytes (9): 50 45 54 52 45 4e 4b 4f 0d

TX ACK: ACK:PETRENKO

TX bytes (13): 41 43 4b 3a 50 45 54 52 45 4e 4b 4f 0a

Verify: OK

Успішний обмін підтверджується тим, що байти RX bytes збігаються з TX hex host, а host отримує ACK:PETRENKO і друкує Verify: OK.

Крок 6. Порівняння і аналіз результатів. Проаналізувати отримані результати і оформити звіт.

ПИТАННЯ ДЛЯ САМОПЕРЕВІРКИ

1. Привести основні характеристики інтерфейсу RS-232C .
2. Привести призначення сигналів інтерфейсу RS-232C (зокрема TXD, RXD, GND).
3. Привести формат даних інтерфейсу RS-232C / UART (старт-біт, біти даних, парність, стоп-біт).
4. Привести формат слова передавального та приймального порту; пояснити позначення 8N1.
5. Привести особливості основних режимів функціонування передавального та приймального портів.
6. Пояснити, яким способом порт переводиться з режиму налаштування в режим основної роботи.
7. Пояснити основні принципи розроблення схеми алгоритму первинного налаштування передавача та приймача.
8. Привести основні елементи схеми алгоритму обміну даними між передавачем і приймачем (відкриття порту, передача повідомлення, прийом кадру, формування підтвердження).
9. Привести основні етапи відлагодження програм передавача та приймача.
10. Пояснити роботу написаних програм передавача (uart_host) і приймача (uart_device_emu); навіщо потрібна віртуальна пара COM і ключ --wait-ack.
11. Додаткові питання (за потреби):
12. Чим відрізняється самоперевірка на loop:// від повного обміну Host↔Device?
13. Навіщо baudrate і формат кадру мають збігатися на обох сторонах лінії?
14. Чим TTL-UART відрізняється від RS-232C за рівнями сигналу та полярністю? Навіщо потрібен перетворювач рівнів (MAX232) і чому не можна з'єднувати виводи мікроконтролера напряду з роз'ємом DB9?
15. Чим RS-485 відрізняється від RS-232 за топологією та дальністю? Навіщо поверх UART часто стоїть Modbus RTU (інвертор / BMS / Home Assistant)?

ЗМІСТ ЗВІТУ

1. Мета роботи.
2. Короткі теоретичні відомості (ролі TX/RX; сигнали TXD/RXD/GND; idle/start; формат кадру UART / 8N1).
3. Схема алгоритму налаштування передавача та приймача (або схема моделі обміну).
4. Хід роботи:
 - параметри варіанту та повідомлення (прізвище);
 - передавач (TX) та його результати роботи;
 - приймач (RX) та його результати роботи;
 - порівняння байтів на TX і RX.
5. Висновки.
6. Текст програми (у додатку).

ЛАБОРАТОРНА РОБОТА № 2

ДОСЛІДЖЕННЯ ГРАФІЧНОГО ПРЕДСТАВЛЕННЯ СИГНАЛІВ ЛІНІЇ ЗВ'ЯЗКУ ТА КОДУВАННЯ NRZI ІНТЕРФЕЙСА УПШ (USB)

МЕТА РОБОТИ: опанування студентом особливостей передачі пакету даних через лінію зв'язку у графічному представленні сигналів для послідовного асинхронного інтерфейса RS-232C (COM-порта / UART) та особливостей формування посимвольних даних у схемі кодування NRZI через інтерфейс УПШ (USB).

Завдання на роботу. Конкретний пакет даних та відповідне повідомлення студенту визначає викладач згідно з [таблицею варіантів](#). Повідомлення – прізвище студента латиницею (літери A–Z). Задаються швидкість обміну даними (baudrate) та формат кадру UART (8N1, 7E1 або 8N2).

Технологія виконання роботи. У лабораторній роботі № 1 формат лінії на стороні host було зафіксовано як 8N1, а акцент зроблено на живому обміні байтами між передавачем і приймачем. У лабораторній роботі № 2 досліджується вже амплітудно-часове представлення сигналу: для заданого повідомлення будуються діаграми UART (старт-біт, біти даних, біт парності за наявності, стоп-біт(и), пауза між символами) та діаграми кодування NRZI, що застосовується у USB 2.0.

Для побудови діаграм використовують програмні засоби курсу: модулі [encoding/uart_plot.py](#), [encoding/usb_nrzi.py](#) а також довідковий графічний інтерфейс [host/signal_gui.py](#) із збереженням рисунків у форматі PNG. Результатом роботи є дві діаграми для всього повідомлення, ручний розрахунок загального часу передавання повідомлення по UART та посимвольний розбір одного символу прізвища (кадр UART і відповідне кодування NRZI).

Кроки виконання:

Крок 1. Визначення вихідних даних. За [варіантом](#) студент фіксує повідомлення (прізвище), baudrate і формат кадру. Приклад варіанту: повідомлення PETRENKO, швидкість 9600 бод, формат 8N1.

Крок 2. Побудова амплітудно-часової діаграми UART. У корні проєкту виконати команду за варіантом, наприклад:

```
python3 -m encoding.uart_plot --message "PETRENKO" --baud 9600 --format 8N1
```

Програма відкриває графік matplotlib для всього повідомлення та виводить розрахований час передачі (Рис. 7).

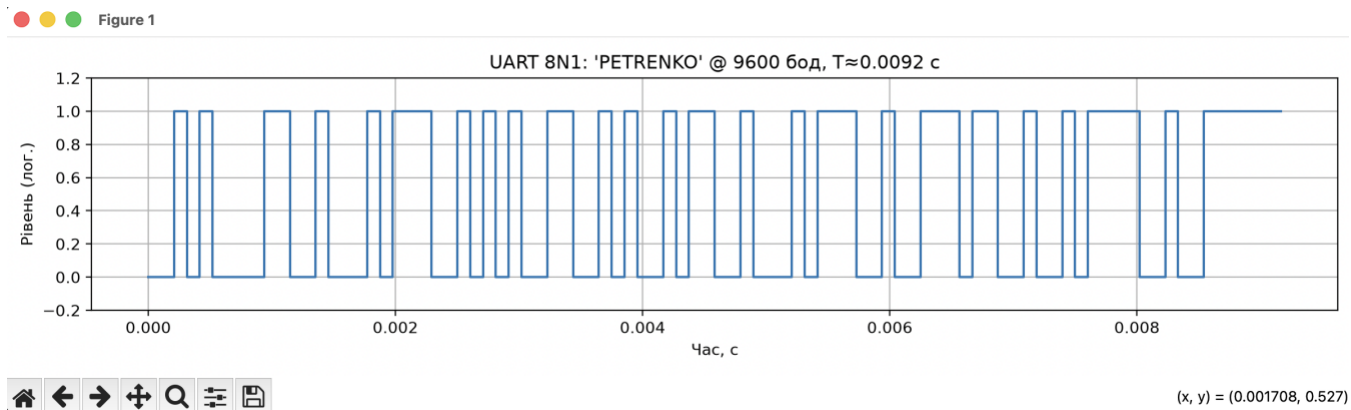


Рис.7. Амплітудно-часова діаграма UART

Отже розрахований час передачі: **0.0092 с.**

Діаграму зберігають у PNG для звіту. На рисунку (або в підписі до нього) обов'язково позначають для одного символу: старт-біт, біти даних, парність (якщо є), стоп-біт(и).

Альтернативно можна використати host/signal_gui.py для задання потрібних параметрів через графічний інтерфейс:

python3 -m host.signal_gui

На Рис. 8 зображено приклад роботи програми.

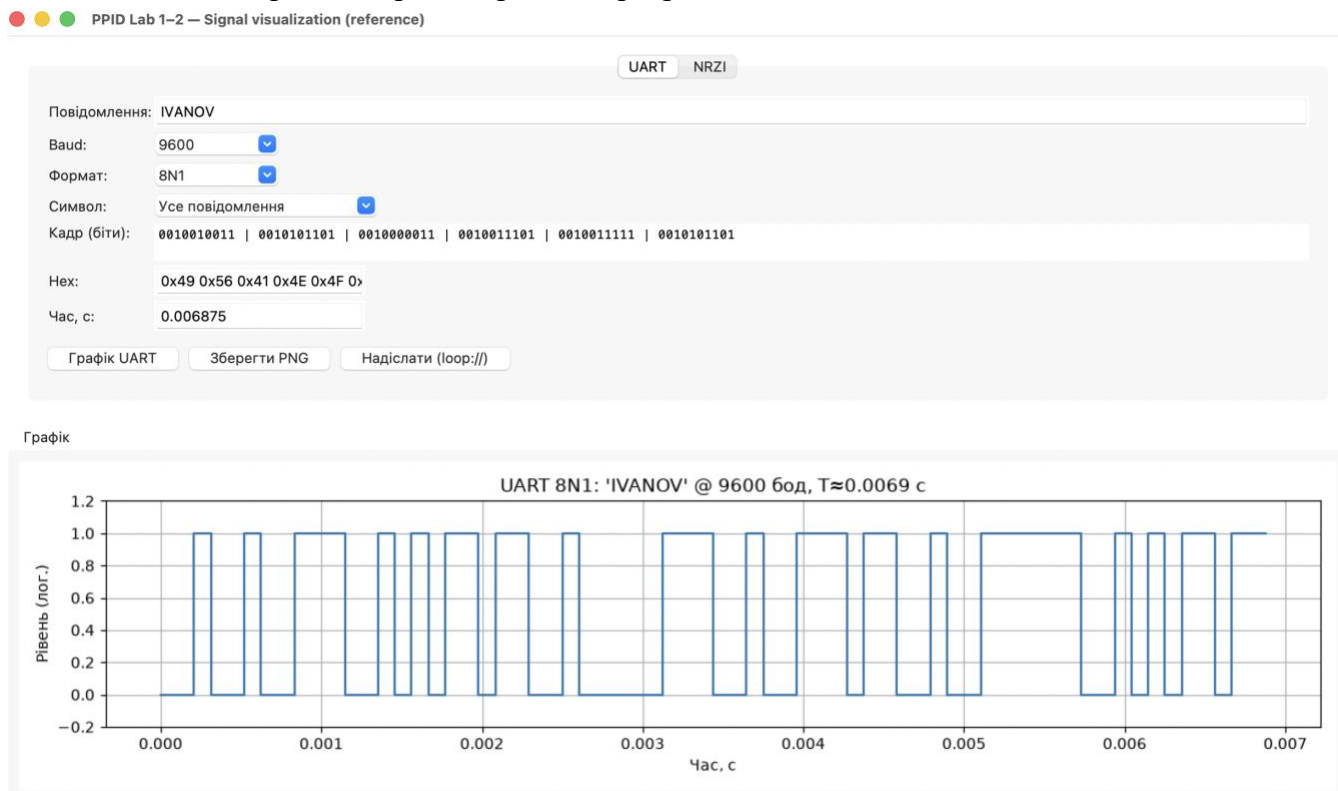


Рис.8. Формування амплітудно-часової діаграми UART для заданого варіанту

Крок 3. Побудова діаграми кодування NRZI. У корні проєкту виконати команду за варіантом, наприклад:

```
python3 -m encoding.usb_nrzi --message "PETRENKO"
```

Очікуваний фрагмент виводу консолі:

Біти (raw): 0101000001000101010101000101001001000101010011100100101101001111

Після bit stuffing:

0101000001000101010101000101001001000101010011100100101101001111

NRZI рівні: [1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0]

Програма відкриває графік matplotlib для всього повідомлення та виводить розрахований час передачі (Рис. 8) NRZI.

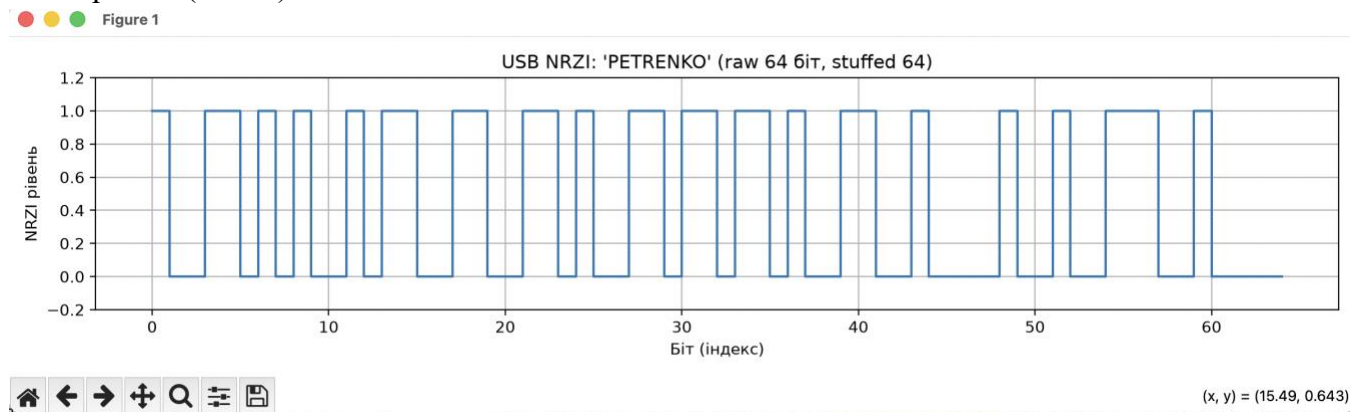


Рис 9. Амплітудно-часова діаграма NRZI

Альтернативно можна використати [host/signal_gui.py](#) для задання потрібних параметрів через графічний інтерфейс:

```
python3 -m host.signal_gui
```

На Рис. 8 зображено приклад роботи програми.

USB NRZI: 'IVANOV' (raw 48 bit, stuffed 48)

The diagram shows a digital signal plot for NRZI encoding. The vertical axis is labeled 'NRZI рівень' (NRZI level) and ranges from -0.2 to 1.2. The horizontal axis is labeled 'Біт (індекс)' (Bit (index)) and ranges from 0 to 50. The signal is a square wave that starts at a high level (1.0) and transitions to a low level (0.0) for each bit of the name 'IVANOV'. The signal returns to high for each subsequent bit. The signal is high for the first 48 bits and then drops to low for the final 2 bits, indicating the end of the data stream.

Тривалість одного такту:

$$T_{\max} = 1 / \text{baudrate}.$$

$$T_{\text{символ}} = 11 \times T_{\text{такт}}$$

Т повідомлення = 8×11

Для першої літери прізвища (наприклад, Р, ASCII 0x50

Приклад кадру 8N1: 0 + 01010000 + 1 = 0010100001.

№ у кадрі	Біт	Роль
1	0	старт
2-9	01010000	дані
10	1	стоп

Далі для тих самих 8 біт даних (без start/stop) застосовують правило NRZI: логічний 0 – зміна рівня, логічна 1 – рівень не змінюється; за потреби враховують bit stuffing після шести одиниць підряд. Результат звіряють із виводом host/signal_gui.py (Рис. 11).

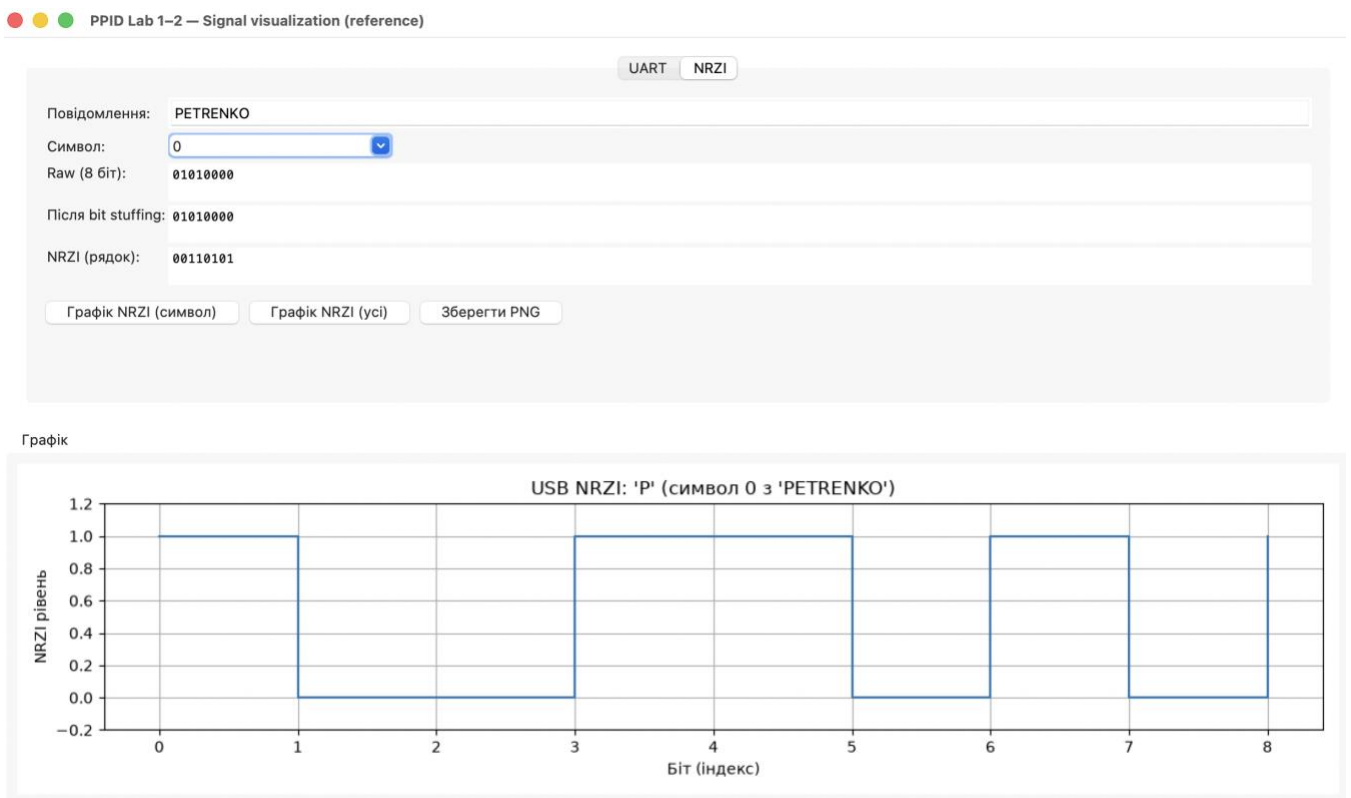


Рис. 11. Посимвольний NRZI аналіз букви “P” прізвища PETRENKO

Крок 6. Оформлення результатів. Проаналізувати отримані результати та сформувати звіт. У звіті необхідно показати скріншоти виконання програм, посимвольний розбір кожної літери прізвища і перевірку результатів.

ПИТАННЯ ДЛЯ САМОПЕРЕВІРКИ

1. Привести співвідношення швидкості обміну інформацією та тривалості одного такту для інтерфейсу RS-232C.

2. Привести сигнальні рівні кодування логічної одиниці та логічного нуля для передавача та приймача.
3. Привести формат даних інтерфейсу RS-232C / UART; пояснити позначення форматів 8N1, 7E1, 8N2.
4. Пояснити, яким способом можна розрахувати загальний час передавання заданого повідомлення.
5. Привести основні характеристики інтерфейсу USB у частині, потрібній для розуміння кодування лінії.
6. Привести особливості системи кодування NRZI.
7. Пояснити призначення процедури bit stuffing у NRZI.
8. Привести основні етапи побудови амплітудно-часової діаграми UART для заданого повідомлення.
9. Привести основні етапи побудови діаграми NRZI для заданого повідомлення.
10. Пояснити роботу програм (uart_plot, usb_nrzi / signal_gui) для заданого пакету даних.
11. Чим відрізняється логічний рівень у програмній моделі від електричного рівня на лінії RS-232?
12. Для якої версії USB застосовується NRZI, а яке кодування використовує SuperSpeed (USB 3.x)?

ЗМІСТ ЗВІТУ

1. Мета роботи.
 2. Короткі теоретичні відомості (кадр UART; NRZI; bit stuffing).
 3. Хід роботи: діаграми UART і NRZI для всього повідомлення (PNG); підписи start/data/parity?/stop; посимвольний розбір кожного символу прізвища.
 4. Розрахунок загального часу передавання заданого повідомлення (UART).
 5. Висновки.
 6. Текст програми (у додатку).
- Демонстрація на комп'ютері програми для заданого пакету даних.

ТЕОРЕТИЧНІ ВІДОМОСТІ ДО ЛАБОРАТОРНОЇ РОБОТИ № 3

Лабораторна робота № 3 призначена для поглибленого вивчення особливостей, характеристик інтерфейсу універсальної послідовної шини, одного із найбільш поширених периферійних інтерфейсів, та отримання навиків розроблення для цього інтерфейсу програмних драйверів, які є складовими компонентами багатьох комп'ютерних систем.

Універсальна послідовна шина УПШ (англ. USB - Universal Serial Bus, аббревіатура читається ю-ес-бі) призначена для з'єднання периферійних пристроїв у периферійну підсистему комп'ютера. Шина USB представляє собою послідовний інтерфейс передачі даних для середньошвидкісних та низькошвидкісних периферійних пристроїв. Для високошвидкісних пристроїв на сьогодні кращим вважається FireWire. USB-кабель представляє собою дві виті пари: по одній парі відбувається передача даних в кожному напрямку (диференціальне включення), а інша пара використовується для живлення периферійного пристрою (+5 В, +3,3В). Завдяки вбудованим лініям живлення, що забезпечують струм до 500 мА, USB часто дозволяє використовувати пристрої без власного блоку живлення (якщо ці пристрої споживають струм потужністю не більше 500 мА). До одного контролера шини USB можна під'єднати до 127 пристроїв через ланцюжок концентраторів (вони використовують топологію «зірка»). На відміну від багатьох інших стандартних з'єднувачів, для USB з'єднувачів характерні довговічність та механічна міцність. Інтерфейс USB є послідовною, напівдуплексною, двонаправленою шиною. Шина дозволяє підключити до ПК до 127 фізичних пристроїв. Кожен фізичний пристрій може, у свою чергу, складатися з декількох логічних (наприклад, клавіатура з вбудованим манипулятором-трекболом). Кабельна розводка USB починається з вузла головного контролера (хост-host). Хост володіє інтегрованим кореневим концентратором (root hub), який надає декілька роз'ємів USB для підключення зовнішніх пристроїв. Потім кабелі йдуть до інших пристроїв USB, які також можуть бути концентраторами, і функціональних компонентів (наприклад, модем або акустична система). Концентратори часто вбудовуються в монітори і клавіатури (які є типовими складеними пристроями). Концентратори можуть містити до семи «витікаючих» портів. Для передачі сигналів шина USB використовує чотирипровідний інтерфейс. Одна пара провідників («+5В» чи «+3,3В» і «загальний») призначена для живлення периферійних пристроїв з навантаженням до 500 мА. Дані передаються по іншій парі («D+» «D-»). Для передачі даних використовується диференціальна напруга до 3 В (з метою зниження впливу шуму) і схема кодування NRZI (що позбавляє від необхідності виділяти додаткову пару провідників під тактовий сигнал). Всі концентратори повинні підтримувати на своїх витікаючих портах пристрої обох типів, не дозволяючи високошвидкісному трафіку досягати низькошвидкісних пристроїв. Високопродуктивні пристрої підключаються за допомогою екранованого кабелю, довжина якого не повинна перевищувати 3 м. Якщо ж пристрій не формує особливих вимог до смуги пропускання, його можна підключити і неекранованим кабелем (який може бути тоншим і гнучкішим). Максимальна довжина кабелю для низькошвидкісних пристроїв – 5 м. Вимоги пристрою до живлення (діаметр провідників, споживана

потужність) можуть зумовити необхідність використання кабелю меншої довжини. Із-за особливостей розповсюдження сигналу по кабелю число послідовно сполучених концентраторів обмежене шістьма і сімома п'ятиметровими відрізками кабелю. Хост дізнається про підключення або відключення пристрою із повідомлення від концентратора (ця процедура називається опитуванням шини - bus enumeration). Потім хост привласнює пристрою унікальну адресу USB (1:127). Після відключення пристрою від шини USB його адреса стає доступною для інших пристроїв. Для індивідуального звернення до конкретних функціональних можливостей складеного пристрою застосовується 4-бітове поле кінцевої крапки. У низькошвидкісних пристроях за кожною функцією закріплюється не більше двох адрес кінцевих крапок: нульова кінцева крапка використовується для конфігурації і визначення стану USB, а також управління функціональним компонентом; а інша крапка - відповідно до функціональних можливостей компоненту. Пристрої з максимальною продуктивністю можуть підтримувати до 16 кінцевих крапок, резервуючи нульову крапку для завдань конфігурації і управління USB. Хост опитує всі пристрої і видає їм дозволу на передачу даних (розсилаючи для цього пакет-маркер - Token Packet). Таким чином, пристрої позбавлені можливості безпосереднього обміну даними - всі дані проходять через хост. Ця умова сильно заважала впровадженню інтерфейсу USB на ринок портативних пристроїв. У результаті в кінці 2001 року було прийнято доповнення до стандарту USB 2.0 - специфікація USB OTG (On-The-Go), призначена для з'єднання периферійних USB-пристроїв один з одним без необхідності підключення до хосту (наприклад, цифрова камера і фотопринтер). Пристрій, підтримуючий USB OTG, здатний частково виконувати функції хоста і розпізнавати, коли він підключений до повноцінного хосту (на основі ПК), чи до іншого периферійного пристрою. Специфікація описує також протокол узгодження вибору ролі хоста при з'єднанні двох USB OTG-пристроїв.

Дані на шині передаються транзакціями, інтервал між якими складає 1 мс. Передбачено чотири типи транзакцій. Передачі, що управляють, використовуються для конфігурації заново підключених пристроїв (наприклад, привласнення ним адреси USB) та їх компонентів. Пристрої з максимальною продуктивністю можуть бути налаштовані на роботу з конфігураційними повідомленнями завдовжки 8, 16, 32 або 64 байти (за умовчанням - 8 байт). Пристрої з низькою продуктивністю в змозі розпізнавати повідомлення, що управляють, завдовжки не більше 8 байт. Групова передача (bulk) використовується для адресної пересилки даних великого об'єму (до 1023 байт). Як приклад можна привести передачу даних на принтер або від сканера. Пристрої з низькою продуктивністю не підтримують цей режим. Передача даних переривання, наприклад, введених з клавіатури даних або відомостей про переміщення миші. Ці дані мають бути передані достатньо швидко для того, щоб користувач не відмітив ніякої затримки. Відповідно до специфікацій час затримки USB складає декілька мілісекунд. Можливі ізохронні передачі (передачі в реальному масштабі часу). Пропускна спроможність і затримка доставки обмовляються до початку передачі даних. До ізохронних даних алгоритми корекції помилок непридатні (оскільки час на повторну їх ретрансляцію перевищує допустимий інтервал затримки). За один сеанс в такому режимі може бути передано до 1023 байт. Пристрої з низькою про-

дуктивністю не підтримують цей режим. Варто також відзначити, що різними виробниками пропонувалися специфікації, що описують інтерфейс різних апаратних реалізацій контролера USB. Фірмою Intel була запропонована специфікація UHCI (Universal Host Controller Interface), яка передбачає надзвичайно просту апаратну реалізацію контролера USB. У межах цієї специфікації основні функції контролю і арбітражу шини покладаються на програмний драйвер. Альтернативна специфікація була запропонована компаніями Compaq, Microsoft і National Semiconductor - OHCI (Open Host Interface). Контролери зі специфікації OHCI володіють уніфікованим абстрактним інтерфейсом, що передбачає апаратну реалізацію більшості функцій, що управляють інтерфейсом, Це полегшує їх програмування.

Для інтерфейсу USB розроблено декілька версій. Версія USB 1.0 представлена в січні 1995 року. Вона забезпечувала наступні технічні характеристики:

- високошвидкісне з'єднання – 12 Мбіт/с;
- максимальна довжина кабеля для високошвидкісного з'єднання – 3 м;
- низькошвидкісне з'єднання – 1,5 Мбіт/с;
- максимальна довжина кабелю для низькошвидкісного з'єднання – 5 м;
- максимальна кількість пристроїв підімкнення (враховуючи) концентратори – 127;
- можливість підключення пристроїв із різними швидкостями обміну інформацією;
- напруга живлення для периферійних пристроїв – 5 В;
- максимальний струм споживання на один пристрій – 500 мА.

Версія USB 1.1 випущено у вересні 1998 року. Виправлені проблеми, виявлені у версії 1.0, в основному пов'язані з концентраторами. Інтерфейс USB 1.1 декларує два режими: 1) низькошвидкісний підканал (пропускна спроможність – 1,5 Мбіт/с), призначений для таких пристроїв, як миші і клавіатури;

2) високопродуктивний канал, що забезпечує максимальну пропускну спроможність 12 Мбіт/с, що може використовуватися для підключення зовнішніх накопичувачів або пристроїв обробки і передачі аудіо- і відеоінформації.

Версія USB 2.0 випущена в квітні 2000 року. USB 2.0 відрізняється від USB 1.1 лише швидкістю передачі, яка зросла, та незначними змінами в протоколі передачі даних для режиму Hi-speed (480 Мбіт/сек). Існує три швидкості роботи пристроїв USB 2.0:

- Low-speed 10–1500 Кбіт/с (використовується для інтерактивних пристроїв: клавіатури, мишки, джойстики);
- Full-speed 0,5–12 Мбіт/с (аудіо/відео пристрої);
- Hi-speed 12–480 Мбіт/с (відео пристрої, пристрої зберігання інформації);

Насправді хоча й швидкість USB 2.0 може досягати 480 Мбіт/с, пристрої типу жорстких дисків чи взагалі будь-які інші носії інформації ніколи не досягають її по шині USB, хоча і могли б. Це можна пояснити доволі просто, шина USB має доволі велику затримку між запитом на передачу інформацію і самою передачею даних (“довгий ring”). Наприклад,

шина FireWire забезпечує максимальну швидкість у 400 Мбіт/с, тобто на 80Мбіт/с менше чим у USB, дозволяє досягнути більшої швидкості обміну даними з носіями інформації.

Технологія USB On-The-Go розширює специфікації USB 2.0 для легкого з'єднання між собою периферійних USB-пристроїв безпосередньо між собою без задіяння комп'ютера. Прикладом застосування цієї технології є можливість підключення фотоапарату напряду до друкарки. Цей стандарт виник через об'єктивну потребу надійного з'єднання особливо поширених USB-пристроїв без застосування комп'ютера, який в потрібний момент може і не виявитися під руками.

Офіційна специфікація протоколу бездротового USB була анонсована в травні 2005 року. Дозволяє організовувати бездротовий зв'язок з високою швидкістю передачі даних до 480 Мбіт/с на відстані 3 м та до 110 Мбіт/с на відстані 10 м. Для бездротового USB часом використовують аббревіатуру WUSB. Розробник протоколу USB-IF віддає перевагу практиці іменування протокол офіційно Certified Wireless USB.

Протокол USB 3.0 орієнтований на передавання сигналів за допомогою оптоволоконного кабелю. USB 3.0 є зворотно сумісним з USB 2.0 та USB 1.1. Створенням USB 3.0 займаються компанії: Intel, Microsoft, Hewlett-Packard, Texas Instruments, NEC и NXP Semiconductors.

Теоретична пікова пропускна здатність інтерфейсу складає 4,8 Гбіт/с. Специфікація периферійної шини USB була розроблена лідерами комп'ютерної і телекомунікаційної промисловості (Compaq, DEC, IBM, Intel, Microsoft, NEC і Northern Telecom) для підключення комп'ютерної периферії поза корпусом ПК з автоматичною автоконфігурацією (Plug&Play). Перша версія стандарту з'явилася в 1996 р. Агресивна політика Intel по впровадженню цього інтерфейсу стимулює поступове зникнення таких низькошвидкісних інтерфейсів, як RS 232C, Access.bus і тому подібне. Проте для високошвидкісних пристроїв строгішими вимогами до продуктивності (наприклад, доступ до видаленого накопичувача або передача оцифрованого відео) конкурентом USB є інтерфейс IEEE 1394.

Інтерфейс регламентує конструктивну сумісність пристроїв, стандартизуючи типи з'єднувачів та розведення сигналів на контактах. Для інтерфейсу стандартизовано два типи з'єднувачів: а та в (див. Рис. 12)

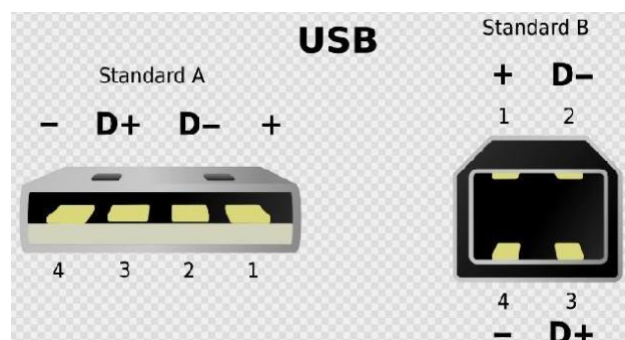


Рис. 12. Схематичне представлення типів з'єднувачів УПШ

У кабелях використовуються провідники згідно з таблицею.

Номер контакту	Позначення	Колір провідника
1	V BUS	червоний
2	D-	білий
3	D+	зелений
4	GND	чорний

GND – ланцюг “корпусу” для живлення периферійних пристроїв, VBus – +5 В, також для ланцюгів живлення. Дані передаються по провідникам D+ і D– диференційно (стан 0 і 1 (в термінології офіційної документації diff0 и diff1 відповідно) визначаються по різниці потенціалів між лініями більше 0,2 В і при умові, що на одній із ліній (D– у випадку diff0 и D+ при diff1) потенціал відносно GND вище 2,8 В. Диференційний спосіб передачі є основним, проте не єдиним (наприклад, при ініціалізації пристрій повідомляє хосту про режим, що підтримується пристроєм (Full-Speed або Low-Speed), підтягуванням однієї із ліній даних до V_BUS через резистор 1.5 кОм (D – для режиму Low-Speed і D+ для режиму Full-Speed, пристрої, що працюють в режимі Hi-Speed, ведуть себе на цій стадії як пристрої в режимі Full-Speed). Також інколи навколо проводу присутня волокниста обмотка для захисту від фізичних пошкоджень.

Інтегруванням напівпровідникової пам’яті та інтерфейсу USB створена флеш пам’ять. Це тип пам’яті, яка може на довготривало зберігати певну інформацію на своїй платі, зовсім не використовуючи живлення. У додаток можна сказати, що флеш пам’ять пропонує високу швидкість доступу до інформації (хоча вона не настільки висока як у DRAM) і кращий опір до кінетичного шоку, ніж у вінчестерах. Ці характеристики пояснюють популярність флеш пам’яті для приладів, що залежать від батарейок. Іншою приманкою флеш пам’яті є те, що коли вона скомпресована в суцільну “карту пам’яті”, стає майже неможливо зруйнувати її стандартними фізичними методами, що дає змогу витримувати високий тиск і кип’ячу воду.

Флеш-пам’ять зберігає інформацію в масиві “комірок”, кожна з яких традиційно зберігає по одному біту інформації. Кожна комірка – це транзистор із плаваючим затвором. Новіші пристрої (інколи їх ще називають багатозарядними пристроями) можуть містити більше, ніж 1 біт в комірці, використовуючи два чи більше рівні електричних зарядів, розташованих при плаваючому затворі комірки. У флеш пам’яті типу NOR кожна комірка схожа на стандартний MOSFET (оксидний напівпровідниковий польовий транзистор), але у ній є не один затвор, а два. Як і будь-який інший польовий транзистор, вони мають контрольний затвор (КЗ), а, окрім нього, ще й інший – плаваючий (ПЗ), замкнений всередині оксидного шару. ПЗ розташований між КЗ і підкладкою. Оскільки ПЗ відокремлений власним заізолюваним шаром оксиду, будь-які електрони, що попадають на нього відразу потрапляють в пастку, що дозволяє зберігати інформацію. Захоплені плаваючим затвором електрони змінюють (практично компенсують) електричне поле контрольного затвору, що змінює порогову напругу ($V_{п}$) затвору. Коли з комірки “зчитують” інформацію, до КЗ прикладають певну напругу, залежно від якої в каналі транзистора протікатиме або не

протікатиме електричний струм. Ця напруга залежить від V_p комірки, яка в свою чергу контролюється числом захоплених плаваючим затвором електронів. Величина порогової напруги зчитується і перекодовується в одиницю чи нуль. Якщо плаваючий затвор може мати кілька зарядових станів, то зчитування відбувається за допомогою вимірювання сили струму в каналі транзистора. Для запису інформації в комірку NOR необхідно зарядити плаваючий затвор. Цього досягають, пропускаючи через канал транзистора сильний струм, при якому виникають гарячі електрони, що мають достатню енергію для подолання оксидного шару. Для очищення плаваючого затвору від електронів (стирання інформації) до контрольного затвору прикладають значну напругу, яка створює сильне електричне поле. Захоплені плаваючим затвором електрони висмоктуються цим полем, тунелюючи через оксидний шар. У приладах з однотипною напругою (теоретично всі чіпи, які доступні нам на сьогоднішній день) ця висока напруга створюється генератором підкачки заряду. Більшість сучасних компонентів NOR пам'яті розділені на чисті сегменти, які часто називають блоками чи секторами. Всі комірки пам'яті в блоці повинні бути очищені одночасно. На жаль, метод NOR може в загальному випадку обробляти лише одну частину інформації типу byte чи word. NAND пам'ять використовує тунельну інжекцію для запису і тунельний випуск для вилучення. NAND'ова флеш пам'ять формує ядро легкого USB інтерфейсу за пам'ятовуючих приладів, які також відомі як USB флешки. Тоді, коли розробники збільшують густину флеш приладів, індивідуальні комірки діляться і кількість електронів у будь-якій комірці стає дуже малою. Парування між суміжними плаваючими затворами може змінити характеристики запису комірки. Нові реалізації, як-от заряджені пастки флеш-пам'яті, намагаються забезпечити кращу ізоляцію між суміжними комірками.

ЛАБОРАТОРНА РОБОТА № 3

РОЗРОБЛЕННЯ ТА ДОСЛІДЖЕННЯ ЕЛЕМЕНТІВ ПРОГРАМНОГО ДРАЙВЕРА ІНТЕРФЕЙСА УПШ (USB)

Мета роботи: опанування студентом технології створення програмного драйвера передавання повідомлення через послідовний інтерфейс УПШ (USB), сканування інтерфейсних портів (у навчальній моделі – mock-перелік пристроїв) та налаштування передачі даних на рівні моделі пакетів і файлової системи

Завдання на роботу. Конкретний пакет даних та відповідне повідомлення студенту визначає викладач згідно [таблиці варіантів](#). Повідомлення – прізвище латиницею (літери A–Z, наприклад IVANOV). Mock-пристрій – зі стовпця «Mock USB» таблиці варіантів. Задається також контекст швидкості обміну.

Технологія виконання роботи. Драйвер передавача має бути налаштований на характеристики, індивідуально задані студенту. Насамперед це довжина інформаційного повідомлення (прізвище) та mock-пристрій варіанту. Спочатку потрібно визначитися зі структурою транзакції. Якщо передавачем є контролер комп'ютера (host), транзакція включає три пакети: маркерний (Token), даних (Data) та підтвердження (Handshake). Маркерний пакет забезпечує синхронізацію пристроїв у моделі, визначає напрям передавання (OUT) і адресу / endpoint. Кількість байт у полі даних пакета Data має бути не меншою за кількість байт заданого повідомлення; у навчальній реалізації [host/usb_transaction.py](#) передбачено мінімум 8 байт payload – коротше прізвище доповнюється нульовими байтами.

Для створення драйвера необхідно розробити відповідну схему алгоритму (побудова кадрів транзакції → сканування переліку пристроїв → вибір пристрою → формування / відкриття тексту повідомлення → запис). Розроблення схеми алгоритму є одним із показників поглибленого розуміння логіки взаємодії пристроїв у периферійній підсистемі на основі інтерфейсу УПШ.

У курсі програмний драйвер реалізується трьома пов'язаними модулями. Приклади реалізації:

Модель транзакції: [host/usb_transaction.py](#). Побудова байтових кадрів OUT: Token → Data → Handshake (hex-дамп).

Сканування портів: [host/usb_scan.py](#). Виведення mock-списку з `fixtures/usb_devices.json` (аналог `lsusb`).

Діалоговий передавач [host/usb_gui.py](#). Виконання «Сканування», вибір пристрою, властивості, відкриття файлу / введення тексту, запис `message.txt`

На Рис. 13 наведено типи з'єднувачів USB (A/B; оглядово USB-C) – для порівняння призначення контактів у коротких теоретичних відомостях звіту.

Типи з'єднувачів USB (вигляд з торця, навчальна схема)



Рис. 13. Типи з'єднувачів USB

На Рис. 14 наведено приклад вікна програмного драйвера передавання даних через USB-інтерфейс.

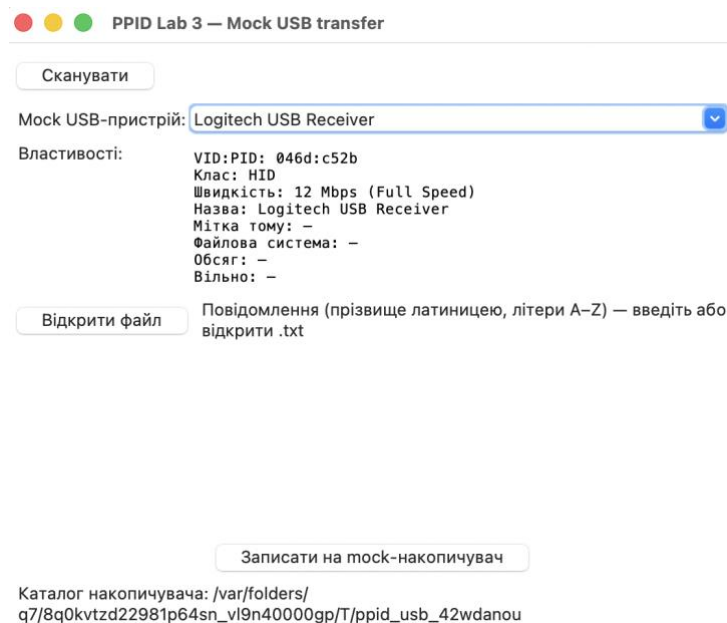


Рис. 14. Вікно програмного драйвера передачі даних через USB-інтерфейс.

Список пристроїв у демонстраційній програмі завжди зчитується з мок даних у JSON – це не реальні USB-порти ПК. Реалізація реального зчитування і запису даних через інтерфейс USB – це додаткова задача у цій лабораторній роботі для отримання максимальної оцінки.

Кроки виконання:

У прикладах використано прізвище PETRENKO і mock-пристрій SanDisk Cruzer (Mass Storage) (як типовий Mass Storage з таблиці варіантів). Під час виконання лабораторної

потрібно підставити власне прізвище (латиницею A–Z) та параметри свого варіанту (номер варіанту, mock-пристрій зі стовпця «Mock USB»).

Крок 1. Визначення вихідних даних. За таблицею варіантів і правилом повідомлення студент фіксує: номер варіанту, mock-пристрій (`mock_usb_name`), за потреби – контекст швидкості обміну для розрахунків, та повідомлення – власне прізвище латиницею. Це саме повідомлення має бути в `--message` для `usb_transaction`, у полі тексту `usb_gui` і в файлі `message.txt` після запису.

Спочатку потрібно визначитися зі структурою транзакції. Якщо передавачем є контролер комп'ютера (`host`), транзакція включає три пакети: маркерний (`Token`), даних (`Data`) та підтвердження (`Handshake`). Кількість байт у полі даних має бути не меншою за довжину повідомлення; у навчальній моделі `usb_transaction` мінімум `payload` – 8 байт (коротше прізвище доповнюється нулями). У звіті також порівняти контакти USB-A та оглядово USB-C.

Крок 2. Побудова моделі транзакції (`Token` → `Data` → `Handshake`). Програма [host/usb_transaction.py](#) формує байтові кадри OUT-транзакції для заданого повідомлення (навчальна модель USB 2.0, не передача по фізичній шині):

```
python3 -m host.usb_transaction --message "PETRENKO"
```

Очікуваний вивід консолі:

Фази: TOKEN → DATA → HANDSHAKE

Пакет 1: e1 81 00

Пакет 2: c3 50 45 54 52 45 4e 4b 4f e7 f4

Пакет 3: d2

Де пакет 1 – `Token` (`PID OUT`); пакет 2 – `Data` (`PID DATA0 c3`, далі ASCII-байти прізвища 50 45 54 52 45 4e 4b 4f = PETRENKO, потім CRC); пакет 3 – `Handshake` (`PID ACK d2`). Кількість байт у полі даних для PETRENKO – 8. Ті самі ASCII-байти з'являлися в лабораторних № 1–2 (TX hex / діаграми).

Крок 3. Сканування інтерфейсних портів (mock-перелік). Програма [host/usb_scan.py](#) виводить навчальний список пристроїв з [fixtures/usb_devices.json](#) (аналог `lsusb`, але не з реальної шини ПК):

```
python3 -m host.usb_scan
```

Очікуваний вивід (фрагмент; у звіті виділити свій рядок варіанту):

Mock USB enumeration (fixtures/usb_devices.json):

У продакшені: lsusb (Linux) або Get-PnpDevice (Windows)

046d:c52b HID @ 12 Mbps (Full Speed) — Logitech USB Receiver
0781:5567 Mass Storage @ 480 Mbps (High Speed) — SanDisk Cruzer (Mass Storage)
303a:1001 CDC @ 12 Mbps (Full Speed) — Espressif USB JTAG/serial (CDC)
1d6b:0002 Hub @ 480 Mbps (High Speed) — Linux Foundation 2.0 root hub
8087:0026 Wireless @ 12 Mbps (Full Speed) — Intel Bluetooth (USB)

Коротко зіставити з реальним переглядом ОС (lsusb / Диспетчер пристроїв): у лабораторній список зчитується з JSON.

Крок 4. Діалоговий драйвер передавача (usb_gui). Запустити графічну програму:

python3 -m host.usb_gui

Послідовність дій у вікні:

1. Натиснути «Сканувати» - оновити mock-список.
2. У ComboBox обрати пристрій варіанту (напр. SanDisk Cruzer (Mass Storage)).
3. Переглянути панель «Властивості» (для Mass Storage типово):

VID:PID: 0781:5567

Клас: Mass Storage

Швидкість: 480 Mbps (High Speed)

Назва: SanDisk Cruzer (Mass Storage)

Мітка тому: PPID_USB

Файлова система: FAT32

Обсяг: 16 ГБ

Вільно: 15.2 ГБ

4. У поле повідомлення ввести прізвище. Приклад - PETRENKO (або відкрити .txt. з цим текстом).
5. Натиснути «Записати на mock-накопичувач» - створюється message.txt у тимчасовій теці ppid_usb_*.
 - a. Перевірити вміст **cat "/path_to/message.txt" -> PETRENKO**
 - b. У звіті додати скриншот вікна і виконання
6. **(Для максимальної оцінки.)** Реалізація драйвера і запис на реальну USB-флешку.
 - a. Реалізувати драйвер;
 - b. Вставити USB-флешку;
 - c. Знайти removable-томи ОС (Windows – знімні диски; macOS – /Volumes/; Linux – /media/\$USER або /run/media/\$USER);
 - d. Зафіксувати мітку тому / формат / розмір обраного носія;
 - e. Записати message.txt з прізвищем на флешку і перевірити результат;

Крок 5. Порівняння і аналіз результатів. Проаналізувати отримані результати і оформити звіт.

ПИТАННЯ ДЛЯ САМОПЕРЕВІРКИ

1. Привести основні характеристики інтерфейсу USB.
2. Привести призначення контактів роз'ємів USB.
3. Привести особливості структур транзакцій інтерфейсу USB.
4. Привести основні етапи відлагодження програмного драйвера.
5. Пояснити роботу написаного програмного драйвера.
6. Пояснити структуру транзакцій інтерфейсу USB .
7. Пояснити основні принципи розроблення схеми алгоритму сканування інтерфейсних портів.
8. Пояснити основні принципи розроблення схеми алгоритму налаштування інтерфейсних портів для передачі даних.
9. Чим mock-список пристроїв відрізняється від `lsusb` або Диспетчера пристроїв?
11. Чим роз'єм USB-C відрізняється від USB-A за кількістю контактів і ролями (host/device)?
12. Чим запис на реальну флешку відрізняється від запису в тимчасову теку для моків `"ppid_usb_"`?

ЗМІСТ ЗВІТУ

1. Мета роботи.
2. Короткі теоретичні відомості.
3. Хід роботи. Навести структуру транзакції інтерфейсу.
4. Висновки.
5. Текст програми (у додатку).
6. Демонстрація програми на комп'ютері.

ТЕОРЕТИЧНІ ВІДОМОСТІ ДО ЛАБОРАТОРНИХ РОБІТ № 4–5

Лабораторні роботи № 4–5 призначені для поглибленого вивчення особливостей, характеристик синхронної послідовної шини I²C, одного із найбільш поширених інтерфейсів підключення периферійних мікросхем, та отримання навиків розроблення для цього інтерфейса програмних драйверів, які є складовими компонентами багатьох комп'ютерних систем.

Інтерфейс I²C за класифікацією послідовний, магістральний, напівдуплексний синхронний інтерфейс. Він реалізовує стандарт обміну даними I²C (Inter-Integrated Circuit – міжмікросхемне з'єднання), який розроблено корпорацією Philips (нині NXP Semiconductors) у 1982 році для зв'язку між інтегральними мікросхемами у межах однієї друкованої плати. На відміну від інтерфейса RS-232C, орієнтованого на зв'язок комп'ютера із зовнішнім периферійним пристроєм, шина I²C призначена насамперед для внутрішньоблочного обміну між мікроконтролером та периферійними мікросхемами: давачами, годинниками реального часу, мікросхемами пам'яті, індикаторами. Відповідно вивчення цього інтерфейсу є обґрунтованим для розуміння побудови периферійної підсистеми вбудованих комп'ютерних систем.

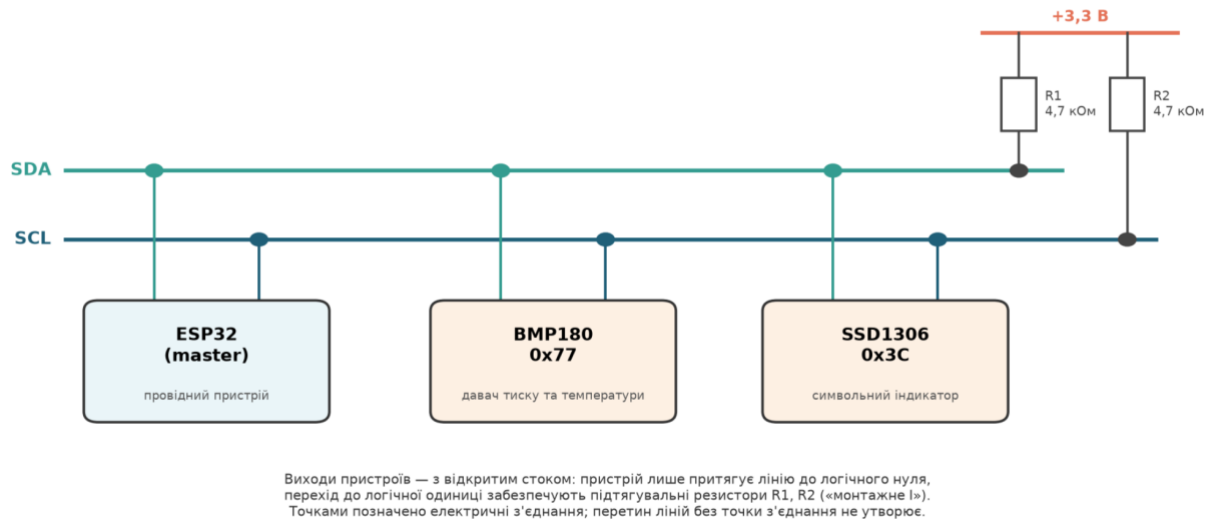
Термін "**послідовний**" означає, що передача даних виконується по одиночному провіднику, а біти передаються послідовно один за другим. У певний момент часу в одному напрямку передається тільки один біт даних.

Термін "**магістральний**" означає, що інтерфейс забезпечує взаємодію багатьох пристроїв, під'єднаних до однієї пари ліній зв'язку. Цим шина I²C принципово відрізняється від радіального інтерфейса RS-232C, який забезпечує взаємодію тільки двох пристроїв. Розрізнення пристроїв на магістралі виконується не окремими лініями зв'язку, а адресою, яка передається на початку кожної транзакції.

Термін "**напівдуплексний**" означає, що інтерфейс забезпечує можливість передачі даних між пристроями в обох напрямках, проте не одночасно, оскільки для даних використовується лише одна сигнальна лінія.

Термін "**синхронний**" означає, що разом з даними по окремій лінії зв'язку передається тактовий сигнал, який формує пристрій-ініціатор обміну. Приймач визначає моменти зчитування бітів за цим тактовим сигналом, а не за власним генератором, як це відбувається в асинхронному інтерфейсі RS-232C. Відповідно шина I²C не потребує попереднього узгодження швидкості обміну між передавачем та приймачем і не використовує стартового, стопового бітів та біта паритету.

Шина I²C використовує дві двонаправлені лінії зв'язку: лінію даних SDA (Serial Data) та лінію тактового сигналу SCL (Serial Clock). Обидві лінії є спільними для всіх пристроїв магістралі. Структурна схема шини з одним провідним та кількома підпорядкованими пристроями показана на рис. 15.

Рис. 15. Структурна схема шини I²C

Пристрій, який ініціює обмін та формує тактовий сигнал, називають провідним (master), а пристрій, який відповідає на звернення за своєю адресою, — підпорядкованим (slave). У лабораторних роботах провідним пристроєм є мікроконтролер ESP32, підпорядкованим — давач атмосферного тиску та температури BMP180 або символьний індикатор SSD1306.

Виходи пристроїв на шині виконані за схемою з відкритим стоком (open drain). Пристрій може лише притягувати лінію до рівня логічного нуля, а перехід до рівня логічної одиниці забезпечують підтягувальні резистори, під'єднані до шини живлення. Опір підтягувальних резисторів обирають у межах приблизно від 2,2 кОм до 10 кОм залежно від швидкості обміну та ємності магістралі, яка не повинна перевищувати 400 пФ. Таке схемотехнічне рішення реалізує логічну функцію "монтажне І": лінія перебуває у стані логічного нуля, якщо хоча б один пристрій притягує її до землі. Це унеможливує пошкодження виходів при одночасній роботі кількох пристроїв та є основою механізму арбітражу магістралі. У симуляторі Wokwi підтягувальні резистори вбудовані у модель шини і на схемі окремо не зображуються.

Кожний підпорядкований пристрій має власну адресу. У базовому варіанті стандарту адреса є семибітовою, що теоретично дає 128 адрес, з яких 16 зарезервовано для службових потреб, тому на одній магістралі можна розрізнити до 112 пристроїв. Адреса задається виробником мікросхеми; частина її розрядів у багатьох мікросхемах може перемикатися зовнішніми виводами, що дозволяє під'єднати до магістралі кілька однотипних пристроїв. Так, давач BMP180 має адресу 0x77, а індикатор SSD1306 — адресу 0x3C. Стандарт передбачає також розширення до десятибітової адресації.

Обмін даними на шині виконується транзакціями. Транзакція розпочинається умовою СТАРТ (START) і завершується умовою СТОП (STOP). Умова СТАРТ формується переходом лінії SDA з високого рівня в низький за високого рівня лінії SCL. Умова СТОП формується зворотним переходом лінії SDA з низького рівня у високий також за високого рівня SCL. В усіх інших випадках рівень на лінії SDA повинен залишатися незмінним

протягом високого рівня SCL, а зміна стану даних відбувається лише за низького рівня тактового сигналу. Часова діаграма транзакції показана на рис. 16.

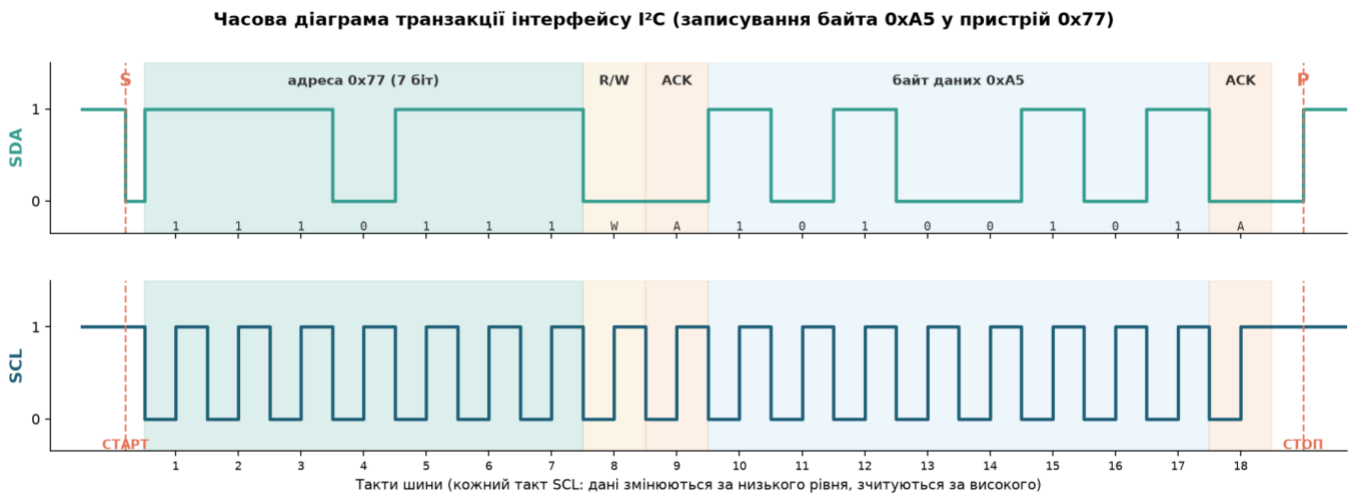


Рис. 16. Часова діаграма транзакції інтерфейсу I²C

Після умови СТАРТ провідний пристрій передає байт, у якому сім старших розрядів становлять адресу підпорядкованого пристрою, а молодший розряд визначає напрям обміну: значення 0 відповідає записуванню даних у підпорядкований пристрій, значення 1 – зчитуванню даних з нього. Кожний переданий байт супроводжується дев'ятим тактом, протягом якого приймач формує біт підтвердження. Якщо приймач притягує лінію SDA до низького рівня, формується підтвердження ACK (Acknowledge), що засвідчує успішне приймання байта. Якщо лінія залишається у високому стані, формується заперечення NACK (Not Acknowledge), яке означає відсутність пристрою із заданою адресою, його неготовність або завершення зчитування з боку провідного пристрою. Відсутність підтвердження після передавання адреси є основною ознакою того, що пристрій на магістралі відсутній.

Більшість периферійних мікросхем має регістрову програмну модель, тобто внутрішній адресний простір регістрів, доступ до яких відбувається за їх номером. Для зчитування регістра застосовують складену транзакцію: провідний пристрій передає адресу підпорядкованого пристрою з ознакою записування та номер потрібного регістра, після чого замість умови СТОП формує повторну умову СТАРТ (repeated START) і передає адресу того самого пристрою вже з ознакою зчитування. Повторна умова СТАРТ не звільняє магістраль, чим унеможливорює втручання іншого провідного пристрою між двома частинами обміну. Часова діаграма зчитування регістра наведена на рис. 17.

Зчитування регістра давача: складена транзакція з повторною умовою СТАРТ

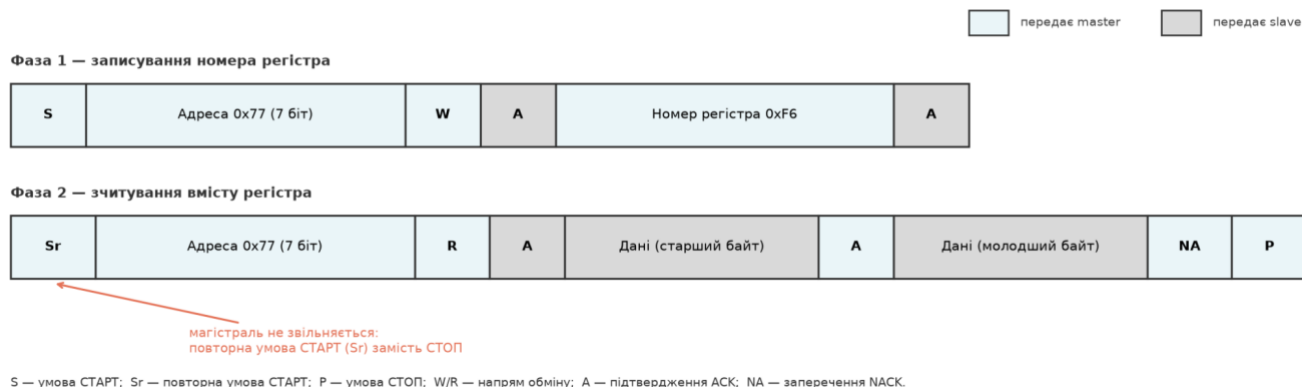


Рис. 17. Часова діаграма зчитування регістра з повторною умовою СТАРТ

Стандарт визначає кілька режимів швидкості обміну, наведених у табл. 2. Швидкість визначається частотою тактового сигналу SCL, який формує провідний пристрій, і не потребує узгодження з підпорядкованим пристроєм так, як це відбувається в асинхронному інтерфейсі. Підпорядкований пристрій, якому потрібен додатковий час на опрацювання даних, може примусово утримувати лінію SCL у низькому стані. Такий механізм називають розтягуванням такту (clock stretching); провідний пристрій при цьому призупиняє обмін до звільнення лінії.

Таблиця 2

Режими швидкості обміну шини I²C

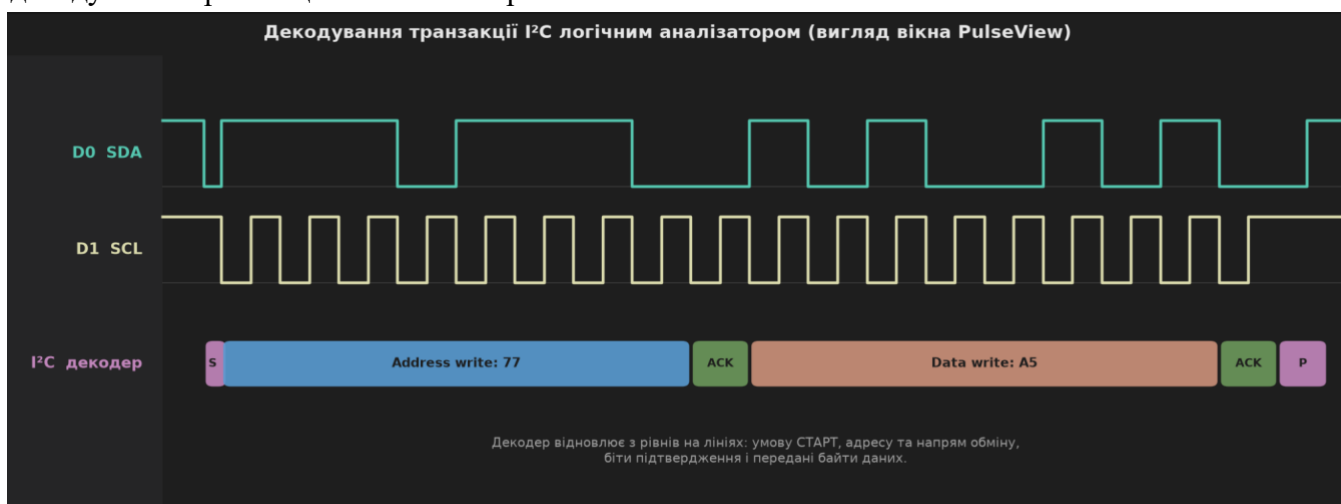
Режим	Максимальна швидкість
Standard mode	100 кбіт/с
Fast mode	400 кбіт/с
Fast mode plus	1 Мбіт/с
High-speed mode	3.4 Мбіт/с

Порівняння основних характеристик послідовних інтерфейсів, які розглядаються у практикумі, наведене в табл. 3. Інтерфейс SPI (Serial Peripheral Interface) також є синхронним послідовним інтерфейсом, проте використовує окремі лінії передавання MOSI та приймання MISO, лінію тактового сигналу SCK та окрему лінію вибору кристала CS для кожного підпорядкованого пристрою. Завдяки повнодуплексному обміну та відсутності адресації SPI забезпечує вищу швидкість, однак потребує більшої кількості ліній зв'язку, що робить його менш зручним за значної кількості пристроїв.

Порівняння послідовних інтерфейсів

Ознака	RS-232c (UART)	I ² C	SPI
Синхроізація	асинхронна	синхронна	синхронна
Кількість ліній даних та такту	2 (TXD, RXD)	2 (SDA, SCL)	4 і більше
Топологія	радіальна	магістральна	магістральна з окремим CS
Режим обміну	дуплексний	напівдуплексний	дуплексний
Адресація	відсутня	7 або 10 біт	лінією CS
Типова швидкість	до 115200 біт/с	100 кбіт/с – десятки Мбіт/с	3,4 Мбіт/с

Для дослідження роботи магістралі застосовують логічний аналізатор – засіб, який фіксує стан ліній зв'язку в часі та відображає його у вигляді часових діаграм. На відміну від амплітудно-часових діаграм, побудованих програмно за моделлю сигналу (лабораторна робота № 2), логічний аналізатор відображає реальні переходи рівнів на лініях SDA та SCL. Засоби декодування дозволяють подати зафіксовану послідовність у вигляді умов СТАРТ і СТОП, адреси, ознаки напрямку обміну, байтів даних та бітів підтвердження. Приклад декодування транзакції показано на рис. 18.

Рис. 18. Декодування транзакції I²C логічним аналізатором.

Лабораторна робота № 5 передбачає об'єднання розглянутих інтерфейсів у складі одного вузла – системи моніторингу, характерної для вбудованих комп'ютерних систем. Структурна схема такого вузла наведена на рис. 19. Периферійний давач під'єднано до мікроконтролера через шину I²C; мікроконтролер із заданою періодичністю опитує давач і передає результати вимірювання через послідовний асинхронний інтерфейс UART до комп'ютера; програма на комп'ютері виконує розбирання прийнятих рядків, збереження результатів у файл та побудову графіка.

Структурна схема вузла моніторингу (лабораторна робота № 5)

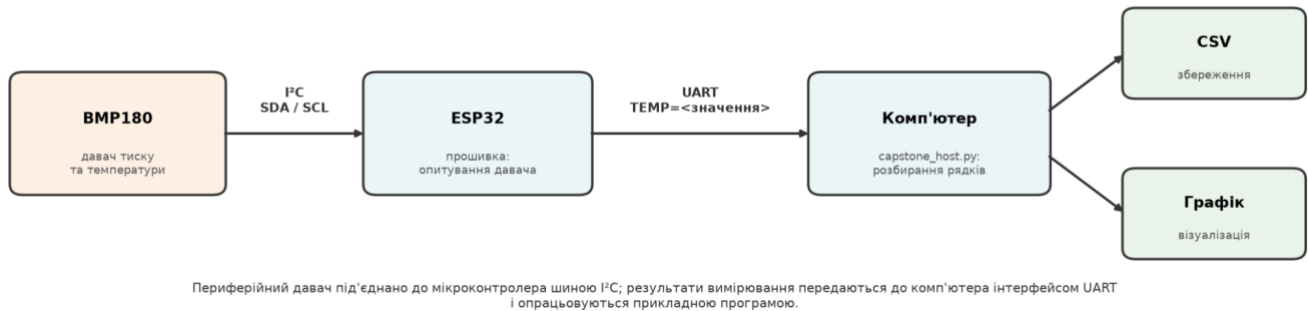


Рис. 19. Структурна схема вузла моніторингу.

Розроблення такого вузла ілюструє поняття рівнів програмного забезпечення периферійної підсистеми. Найнижчий рівень утворює доступ до регістрів мікросхеми давача через апаратний контролер шини; наступний рівень – програмний драйвер пристрою, який приховує особливості регістрової моделі та надає прикладній програмі результат вимірювання у фізичних величинах; вище розташований рівень передавання даних каналом зв'язку, який визначає формат телеметричного повідомлення; найвищий рівень – прикладна програма, що опрацьовує та зберігає отримані дані. Розподіл функцій за рівнями наведено на рис. 20.

Рівні програмного забезпечення вузла моніторингу



Рис. 20. Рівні програмного забезпечення вузла моніторингу.

В операційних системах сімейства Linux доступ до розглянутих інтерфейсів уніфіковано у вигляді файлів пристроїв: магістраль I²C подається файлами `/dev/i2c-*`, послідовні порти – файлами `/dev/tty*`. Прикладна програма звертається до них через стандартні системні виклики, а безпосередню взаємодію з апаратурою виконує драйвер у просторі ядра. Такий поділ на простір користувача та простір ядра є типовим для сучасних комп'ютерних систем і пояснює, чому операції записування даних у файл на накопичувачі не є розробленням драйвера периферійного пристрою.

Обов'язковою умовою працездатності вузла є узгодження формату телеметричного повідомлення між програмою мікроконтролера та програмою комп'ютера. Формат визначає ознаку початку рядка, роздільник, кількість значущих цифр та символи завершення рядка. Невідповідність формату призводить до того, що фізично правильно передані дані не можуть бути розібрані приймальною програмою, що є характерною помилкою під час відлагодження подібних систем.

ЛАБОРАТОРНА РОБОТА № 4

ДОСЛІДЖЕННЯ ШИНИ I2C ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ДРАЙВЕРА ПЕРИФЕРІЙНОГО ДАВАЧА

Мета роботи: опанування студентом принципів роботи синхронної двопровідної магістральної шини I2C, адресації пристроїв master–slave, технології створення програмного драйвера доступу до регістрів периферійної мікросхеми та дослідження реальних сигналів магістралі логічним аналізатором на платформі ESP32 у симуляторі Wokwi.

Завдання на роботу: Тип периферійної мікросхеми студенту визначає викладач згідно таблиці [варіантів](#). Для варіантів з індикатором (4, 6, 9) замість давача використовується символьний OLED-індикатор SSD1306 (адреса 0x3C), на який потрібно вивести повідомлення – прізвище латиницею (літери A–Z, наприклад IVANOV). Задається також частота тактового сигналу магістралі; типове значення – 100 кГц, що відповідає режиму Standard mode.

Технологія виконання роботи. Програмний драйвер має бути налаштований на характеристики, індивідуально задані студенту. Насамперед це тип периферійної мікросхеми та її адреса на магістралі, а для варіантів з індикатором – також інформаційне повідомлення (прізвище). Спочатку потрібно визначитися зі структурою транзакцій, які виконує провідний пристрій. У цій роботі їх дві. Перша – сканування магістралі: провідний пристрій для кожної можливої адреси формує умову СТАРТ і байт адреси з ознакою записування, після чого перевіряє, чи надійшло підтвердження АСК. Наявність підтвердження означає, що пристрій із такою адресою присутній на шині, відсутність – що адреса вільна. Жодного байта даних при цьому не передається. Друга – складена транзакція зчитування регістра: провідний пристрій передає адресу підпорядкованого пристрою з ознакою записування та номер потрібного регістра, після чого замість умови СТОП формує повторну умову СТАРТ і передає ту саму адресу вже з ознакою зчитування, після чого приймає байти даних. Повторна умова СТАРТ не звільняє магістраль і унеможливорює втручання іншого провідного пристрою між двома частинами обміну.

Далі визначають параметри магістралі: виводи мікроконтролера, до яких під'єднано лінії SDA і SCL, та частоту тактового сигналу. У навчальному проєкті лінію даних SDA під'єднано до виводу GPIO21, лінію тактового сигналу SCL – до виводу GPIO22, частоту задано 100 кГц. Виходи пристроїв виконано за схемою з відкритим стоком, тому перехід ліній до рівня логічної одиниці забезпечують підтягувальні резистори; у моделі симулятора вони вбудовані в шину і на схемі окремо не зображуються.

Для створення драйвера необхідно розробити відповідну схему алгоритму (ініціалізація контролера шини → сканування магістралі та перевірка наявності пристрою за заданою адресою → зчитування калібрувальних коефіцієнтів мікросхеми → циклічне запускання вимірювання і приймання результату → перетворення кодів у фізичні величини → виведення). Розроблення схеми алгоритму є одним із показників поглибленого розуміння логіки взаємодії пристроїв у периферійній підсистемі на основі магістрального послідовного інтерфейсу.

У курсі програмний драйвер реалізується трьома пов'язаними файлами.

Приклади реалізації (для варіантів з датчиком тиску):

Опис апаратної частини: [diagram.json](#). Плата ESP32, мікросхема давача BMP180 (адреса 0x77) та під'єднаний до ліній SDA/SCL логічний аналізатор.

Драйвер пристрою: [bmp180.py](#). Зчитування з регістрів, починаючи з адреси 0xAA, одинадцять калібрувальних коефіцієнтів мікросхеми; записування коду команди в регістр керування 0xF4; витримування часу перетворення; зчитування результату з регістра 0xF6; повернення температури у градусах Цельсія та тиску у паскалях.

Прикладна програма: [main.py](#). Створення об'єкта контролера шини, сканування магістралі, періодичне звертання до драйвера і виведення результату у Serial Monitor.

Serial Monitor – це текстовий термінал, у якому відображається все, що мікроконтролер передає інтерфейсом UART; у симуляторі це нижня панель вікна, а на реальній платі його роль виконує термінальна програма, під'єднана до віртуального COM-порту через перетворювач USB-UART. У файлі `diagram.json` термінал описано як компонент схеми, під'єднаний до виводів TX і RX мікроконтролера перехресно, тому оператор `print` не є виведенням у консоль: рядок фізично передається лінією як послідовність старт-стопних кадрів, по одному на кожний символ. Serial Monitor показує готовий текст, переданий лінією TXD, тоді як логічний аналізатор фіксує рівні напруги на лініях SDA і SCL магістралі I2C.

Логічний аналізатор у симуляторі записує рівні на дротах, але не відображає осцилограму у вікні браузера. Після зупинення симуляції формується файл `wokwi-logic.vcd` у форматі **Value Change Dump (VCD)**, який відкривають у зовнішній програмі. Реалізація зчитування сигналів реальним апаратним логічним аналізатором – це додаткова задача у цій лабораторній роботі для отримання максимальної оцінки.

Кроки виконання:

У прикладах використано прізвище PETRENKO і давач BMP180 (як типовий давач температури та тиску з таблиці варіантів). Під час виконання лабораторної потрібно підставити власне прізвище (латиницею A–Z) та параметри свого варіанту (номер варіанту, тип периферійної мікросхеми).

Крок 1. Визначення вихідних даних. За таблицею варіантів студент фіксує: номер варіанту, тип периферійної мікросхеми та її адресу на магістралі, частоту тактового сигналу і відповідний режим швидкості обміну, а для варіантів з індикатором – також повідомлення, тобто власне прізвище латиницею. Це саме повідомлення має бути у змінній SURNAME файла `main.py` і на екрані індикатора після запуску.

Спочатку потрібно визначитися зі структурою транзакцій. У звіті навести обидві: сканування магістралі (умова СТАРТ, байт адреси з ознакою записування, аналіз підтвердження ACK, умова СТОП) та складене зчитування регістра (адреса з ознакою записування, номер регістра, повторна умова СТАРТ, адреса з ознакою зчитування, байти даних, заперечення NACK від провідного пристрою, умова СТОП). Указати частоту тактового сигналу і відповідний режим за стандартом. У звіті також порівняти інтерфейси

I2C та SPI за синхронізацією, кількістю ліній, топологією, способом адресації та швидкістю.

Крок 2. Складання проєкту в симуляторі. Відкрити середовище Wokwi MicroPython ESP32 (<https://wokwi.com/projects/new/micropython-esp32>) і створити три файли, вставивши їх вміст із додатка до методичних вказівок:

[main.py](#). – прикладна програма; [diagram.json](#)– опис схеми (плата, давач, логічний аналізатор); [bmp180.py](#) – драйвер давача.

Для варіантів з індикатором необхідно обрати [main.py](#) і [diagram.json](#) проєкту з індикатором та драйвер [ssd1306.py](#), а у файлі main.py замінюють значення змінної SURNAME на власне прізвище. Драйвер потрібно завантажити із репозиторію курсу.

Драйвери потрібно окремо створити як новий файл у проєкті wokwi.

При наявності фізичного ESP32 і певного датчика, можна продемонструвати роботу програми на ньому.

Крок 3. Сканування магістралі та зчитування давача. Запустити симуляцію (Start Simulation) і відкрити Serial Monitor (Рис 21.)

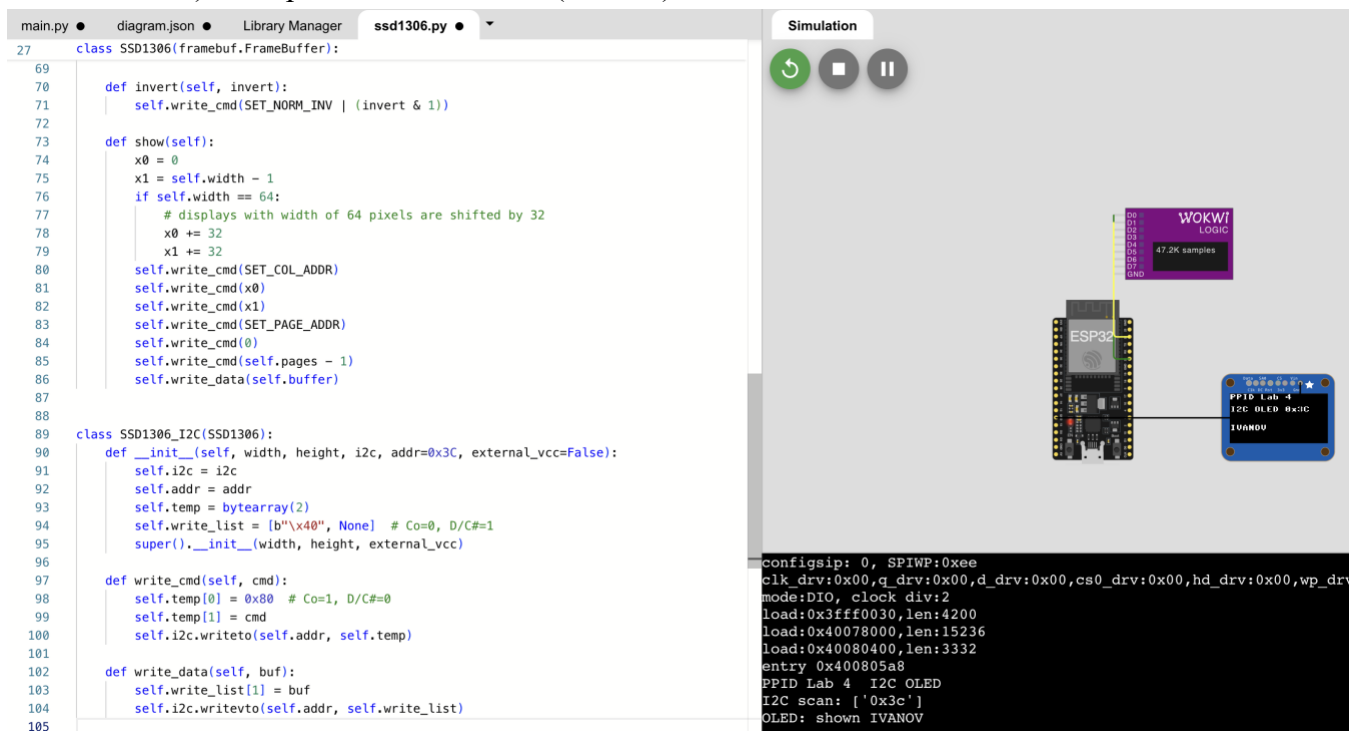


Рис. 21. Приклад симуляції OLED дисплею у Wokwi.

Очікуваний вивід консолі:

PPID Lab 4 I2C OLED

I2C scan: ['0x3c']

OLED: shown IVANOV

Для варіантів з сенсором очікуваний вивід

PPID Lab 4 – I2C sensor

I2C scan: ['0x77']

TEMP=24.0 PRESS=101325.0

TEMP=24.0 PRESS=101325.0

Де рядок I2C scan – результат сканування магістралі: список адрес, від яких надійшло підтвердження АСК. Наявність у списку значення 0x77 підтверджує, що давач присутній на шині та відповідає за своєю адресою. Рядки TEMP і PRESS – результат роботи драйвера: температура у градусах Цельсія і тиск у паскалях, обчислені з кодів регістрів за калібрувальними коефіцієнтами мікросхеми.

У звіті навести рядок сканування, кілька рядків результатів вимірювання та скриншот вікна Serial Monitor, а для варіантів з індикатором – також скриншот екрана з виведеним прізвищем.

Крок 4. Перевірка незалежності прикладної програми від апаратури. Для варіантів з сенсором - під час роботи симуляції клацнути на зображенні давача на схемі – з'являться повзунки temperature і pressure. Змінити їх положення і переконатися, що значення у Serial Monitor оновилися без будь-яких змін у програмі. Це демонструє, що прикладна програма оперує результатом вимірювання у фізичних величинах, а не станом ліній магістралі: перетворення виконує драйвер. У звіті навести два скриншоти – до і після зміни значень.

Крок 5. Дослідження магістралі логічним аналізатором. Переконатися, що під час симуляції на логічному аналізаторі зростає лічильник відліків, після чого натиснути Stop – браузер завантажить файл **wokwi-logic.vcd**. Відкрити файл у програмі PulseView: Open → Import Value Change Dump data..., у діалозі імпорту встановити Downsampling factor = 50. Додати декодер I2C (кнопка Add protocol decoder), указавши канали SDA = D0 (GPIO21) і SCL = D1 (GPIO22).

У звіті навести власний скриншот із позначеними умовою СТАРТ, байтом адреси 0x77, бітом підтвердження АСК та умовою СТОП. Пояснити, чому у стані спокою обидві лінії перебувають на високому рівні та чому активність на магістралі має вигляд окремих пачок переходів, розділених паузами. Пояснити відмінність цього запису від амплітудно-часової діаграми лабораторної роботи № 2: логічний аналізатор відображає реальні переходи рівнів на лініях, а не програмно побудовану модель сигналу.

За неможливості встановити PulseView потрібно використати веб-переглядач Surfer (<https://app.surfer-project.org/>), перетягнувши файл у вікно браузера і додавши сигнали D0 і D1. Декодера інтерфейсу I2C ця програма не має, тому адресу пристрою у звіті підтверджують результатом сканування магістралі з Serial Monitor.

Крок 6. Порівняння і аналіз результатів. Проаналізувати отримані результати і оформити звіт.

ПИТАННЯ ДЛЯ САМОПЕРЕВІРКИ

1. Привести класифікацію інтерфейсу I2C та пояснити зміст термінів «послідовний», «магістральний», «напівдуплексний», «синхронний».

2. Привести призначення ліній SDA і SCL та пояснити роль провідного і підпорядкованого пристроїв.
3. Пояснити, навіщо виходи пристроїв виконано за схемою з відкритим стоком, та яку функцію виконують підтягувальні резистори.
4. Привести особливості структур транзакцій інтерфейсу I2C: умови СТАРТ і СТОП, байт адреси, ознака напрямку обміну, біти підтвердження ACK і NACK.
5. Пояснити, як виконується зчитування регістра давача та навіщо застосовують повторну умову СТАРТ.
6. Пояснити, як визначити адресу пристрою на магістралі та за якою ознакою виявляють його відсутність.
7. Привести основні етапи відлагодження програмного драйвера.
8. Пояснити роботу написаного програмного драйвера.
9. Пояснити основні принципи розроблення схеми алгоритму сканування магістралі.
10. Пояснити основні принципи розроблення схеми алгоритму зчитування регістрів периферійної мікросхеми.
11. Пояснити механізм розтягування такту (clock stretching) і привести режими швидкості обміну інтерфейсу I2C.
12. Чим інтерфейс I2C відрізняється від SPI за кількістю ліній, способом адресації та швидкістю? Чому для під'єднання давачів у межах плати частіше обирають I2C?
13. Чим інтерфейс I2C відрізняється від RS-232C за синхронізацією та топологією?
14. Що показує логічний аналізатор на лініях SDA і SCL під час адресації та чим цей запис відрізняється від діаграми, побудованої програмно у лабораторній роботі № 2?
15. Які рівні програмного забезпечення реалізують файли main.py, bmp180.py та diagram.json навчального проєкту?

ЗМІСТ ЗВІТУ

1. Мета роботи.
2. Короткі теоретичні відомості.
3. Хід роботи.
4. Висновки.
5. Текст програми (у додатку).
6. Демонстрація програми на комп'ютері.

ЛАБОРАТОРНА РОБОТА № 5

РОЗРОБЛЕННЯ ТА ДОСЛІДЖЕННЯ ІНТЕГРОВАНОГО ВУЗЛА МОНІТОРИНГУ КОМП'ЮТЕРНОЇ СИСТЕМИ

Мета роботи: опанування студентом технології інтеграції магістральної шини I2C, послідовного асинхронного інтерфейсу UART та рівня зберігання даних у складі єдиного вузла моніторингу; набуття навичок розроблення багаторівневого програмного забезпечення периферійної підсистеми – від драйвера мікросхеми давача до прикладної програми опрацювання телеметрії.

Завдання на роботу: Інтервал опитування давача та формат телеметричного рядка студенту визначає викладач згідно [таблиці варіантів](#). Типові значення інтервалу – 200, 500 або 1000 мс; типовий формат рядка – TEMP=<значення> із завершенням символами повернення каретки та переведення рядка. Швидкість обміну і формат кадру інтерфейсу UART – ті самі, що задані студенту в лабораторній роботі № 1.

Технологія виконання роботи. Вузол моніторингу складається з трьох частин: периферійної мікросхеми давача, під'єднаної до мікроконтролера магістраллю I2C; програми мікроконтролера, яка з заданою періодичністю опитує давач і передає результат вимірювання інтерфейсом UART; прикладної програми комп'ютера, яка приймає рядки телеметрії, зберігає їх у файл і будує графік. Спочатку потрібно визначитися з розподілом функцій між цими частинами та з форматом телеметричного повідомлення, який є угодою між прошивкою мікроконтролера і програмою комп'ютера.

Формат телеметричного повідомлення визначає ознаку початку рядка, роздільник, кількість значущих цифр і символи завершення рядка. У навчальному проєкті прошивка формує рядок виду TEMP=22.1 з одним знаком після коми. Програма комп'ютера відшукує в кожному прийнятому рядку цю ознаку за регулярним виразом і вилучає числове значення; рядки, які не відповідають формату, пропускаються без аварійного завершення програми. Невідповідність формату призводить до того, що фізично правильно передані дані не можуть бути розібрані приймальною програмою – це характерна помилка під час відлагодження подібних систем, оскільки зовні вона виглядає як відсутність зв'язку, хоча канал передавання справний.

Слід звернути увагу на те, що програма мікроконтролера використовує два різні порти UART. Порт UART0 задіяно інтерпретатором MicroPython для стандартного потоку виведення: усе, що виводить оператор print, потрапляє у Serial Monitor – текстовий термінал, під'єднаний у схемі до виводів TX і RX мікроконтролера перехресно. Порт UART1, налаштований окремо на швидкість 9600 біт/с з виводом передавача на GPIO17, є власне каналом телеметрії: саме в нього метод write передає сформований рядок. Прошивка навмисно дублює рядок в обидва порти. У каналі UART1 на протилежному кінці лінії в умовах симулятора приймача немає, тому джерелом даних для прикладної програми слугує копія, збережена з Serial Monitor. На реальному обладнанні дублювання не потрібне: комп'ютер під'єднують безпосередньо до ліній порту UART1 через перетворювач USB-UART, і прикладна програма читає дані з віртуального COM-порту.

Логічний аналізатор у схемі під'єднано до трьох ліній: до ліній SDA і SCL магістралі I2C (канали D0 і D1) та до лінії передавача порту UART1 (канал D2). Це дозволяє в одному записі спостерігати обидва інтерфейси одночасно: короткі пакети переходів на лініях

магістралі, що відповідають опитуванню давача, і старт-стопні кадри на лінії UART, що відповідають передаванню телеметричного рядка. Періодичність цих груп визначається заданим інтервалом опитування. Саме цей запис є основним експериментальним підтвердженням того, що обидва інтерфейси працюють у складі одного вузла узгоджено.

Для створення вузла необхідно розробити схеми алгоритмів обох програм. Для прошивки мікроконтролера: ініціалізація контролера магістралі та порту UART → зчитування калібрувальних коефіцієнтів давача → у циклі: запускання вимірювання, приймання результату, перетворення кодів у фізичну величину, формування телеметричного рядка, передавання його в порт → витримування заданого інтервалу. Для прикладної програми комп'ютера: відкриття джерела даних → построкове читання → розпізнавання ознаки та вилучення числового значення → накопичення масиву результатів → записування у файл формату CSV → побудова графіка. Розроблення схем алгоритмів є одним із показників поглибленого розуміння розподілу функцій між рівнями програмного забезпечення периферійної підсистеми.

У курсі вузол моніторингу реалізується чотирма пов'язаними файлами. Приклади реалізації:

Опис апаратної частини: diagram.json. Плата ESP32, мікросхема давача BMP180 (адреса 0x77) та логічний аналізатор, під'єднаний до ліній магістралі і до лінії передавача телеметрії.

Драйвер пристрою: bmp180.py. Той самий драйвер, що в лабораторній роботі № 4: доступ до регістрів мікросхеми, калібрування, повернення температури у градусах Цельсія.

Прошивка мікроконтролера: main.py. Ініціалізація магістралі та порту UART, циклічне опитування давача із заданим інтервалом INTERVAL_MS, формування та передавання телеметричного рядка.

Прикладна програма комп'ютера: host/capstone_host.py. Розбирання рядків телеметрії, записування результатів у файл CSV, побудова графіка та експорт на mosk-накопичувач.

На Рис. 22 наведено структурну схему вузла моніторингу – для пояснення призначення кожного елемента та ролі обох інтерфейсів у коротких теоретичних відомостях звіту.



Рис. 22. Структурна схема вузла моніторингу.

На Рис. 23 наведено рівні програмного забезпечення вузла: апаратний рівень, драйвер пристрою, рівень передавання даних та прикладна програма:

Рівні програмного забезпечення вузла моніторингу



Рис. 23. Рівні програмного забезпечення вузла моніторингу.

Нижчі рівні приховують особливості апаратури, тому прикладна програма оперує результатом вимірювання, а не станом ліній магістралі. В операційних системах сімейства Linux доступ до розглянутих інтерфейсів уніфіковано у вигляді файлів пристроїв: магістраль I2C подається файлами `/dev/i2c-*`, послідовні порти – файлами `/dev/tty*`. Прикладна програма звертається до них через стандартні системні виклики, а безпосередню взаємодію з апаратурою виконує драйвер у просторі ядра. Тому записування даних у файл на накопичувачі не є розробленням драйвера периферійного пристрою. Реалізація приймання телеметрії безпосередньо з послідовного порту в реальному часі – це додаткова задача у цій лабораторній роботі для отримання максимальної оцінки.

Кроки виконання:

У прикладах використано інтервал опитування 500 мс і давач BMP180. Під час виконання лабораторної потрібно підставити параметри свого варіанту (номер варіанту, інтервал опитування, швидкість обміну і формат кадру інтерфейсу UART).

Крок 1. Визначення вихідних даних. Визначення вихідних даних. За таблицею варіантів студент фіксує: номер варіанту, інтервал опитування давача, швидкість обміну та формат кадру інтерфейсу UART, а також формат телеметричного рядка. У звіті навести структурну схему вузла (Рис. 18) з позначенням, який інтерфейс з'єднує кожну пару елементів, і зазначити, які саме параметри є предметом угоди між прошивкою та програмою комп'ютера.

Крок 2. Складання проєкту в симуляторі. Відкрити середовище Wokwi MicroPython ESP32 (<https://wokwi.com/projects/new/micropython-esp32>) і створити три файли, вставивши їх вміст із додатка до методичних вказівок:

[main.py](#). – прошивка вузла;

[diagram.json](#) – опис схеми;

[bmp180.py](#) – драйвер давача.

У файлі `main.py` виставити значення змінної `INTERVAL_MS`, що дорівнює інтервалу опитування свого варіанту, а параметри порту `UART1` – відповідно до заданої швидкості обміну.

При наявності фізичного ESP32 і певного датчика, можна продемонструвати роботу програми на ньому.

Крок 3. Накопичення телеметрії. Запустити симуляцію (Start Simulation) і відкрити Serial Monitor.

Очікуваний вивід консолі:

PPID Lab 5 — Capstone node

I2C: ['0x77']

TEMP=22.1

TEMP=22.3

TEMP=22.5

Де рядок `I2C` – результат сканування магістралі, який підтверджує наявність давача за адресою `0x77`; наступні рядки – телеметрія, що надходить із заданим інтервалом. Під час роботи симуляції клацнути на зображенні давача і кілька разів змінити положення повзунка `temperature`, щоб у записі були різні значення – графік буде змістовнішим. Накопичений вивід зберегти у текстовий файл `my_log.txt`. У файлі мають бути рядки саме заданого формату, без зайвих символів перед ознакою `TEMP=`.

Крок 4. Дослідження обох інтерфейсів логічним аналізатором. Натиснути Stop – браузер завантажить файл `wokwi-logic.vcd`. Відкрити його у програмі PulseView (Open → Import Value Change Dump data..., Downsampling factor = 50) і додати канали D0 (SDA), D1 (SCL) та D2 (лінія передавача UART1). До каналів D0 і D1 під'єднати декодер I2C, до каналу D2 – декодер UART, указавши задану швидкість обміну і формат кадру.

У звіті навести скріншот, на якому в межах одного інтервалу опитування видно пачку транзакцій магістралі I2C і наступну за нею послідовність кадрів UART. Пояснити, чому ці групи чергуються з періодичністю, що дорівнює заданому інтервалу, та зіставити кількість кадрів UART з довжиною телеметричного рядка у символах.

Крок 5. Опрацювання телеметрії прикладною програмою.. кореня репозиторію запустити програму комп'ютера, указавши збережений файл:

```
python3 -m host.capstone_host --input my_log.txt --plot capstone_plot.png --csv readings.csv
```

Очікуваний вивід:

Знайдено зчитувань: 10
CSV: readings.csv
Графік: capstone_plot.png

Файл **readings.csv** містить заголовок і по одному рядку на кожне розпізнане зчитування, де перше поле – порядковий номер рядка у вихідному записі, друге – значення температури:

```
sample,temp_c
2,22.1
3,22.3
4,22.5
```

У звіті навести фрагмент файла **my_log.txt**, вміст або таблицю файла **readings.csv** та графік **capstone_plot.png**. Пояснити, чому нумерація зчитувань починається не з нуля.

Крок 6. Експорт результатів на накопичувач. Повторити запуск програми з додатковим параметром `--export-usb`. Копіювання файла CSV у тимчасовий каталог `mosc-` накопичувача виконується засобами файлової системи – так само, як у лабораторній роботі № 3. У звіті пояснити, що ця операція належить до рівня прикладної програми і не є розробленням драйвера накопичувача.

Крок 7. (Для максимальної оцінки.) Приймання телеметрії в реальному часі.

- a. Реалізувати приймання рядків безпосередньо з послідовного порту засобами бібліотеки `pyserial`, без проміжного збереження у файл;
- b. Використати віртуальну пару портів з лабораторної роботи № 1 або реальний перетворювач USB-UART;
- c. Забезпечити побудову графіка в міру надходження даних;
- d. Порівняти отриману реалізацію з опрацюванням збереженого запису і пояснити, які додаткові задачі виникають у режимі реального часу.

Крок 8. Порівняння і аналіз результатів. Проаналізувати отримані результати і оформити звіт.

ПИТАННЯ ДЛЯ САМОПЕРЕВІРКИ

1. Привести структурну схему вузла моніторингу та пояснити призначення кожного її елемента.
2. Які інтерфейси використано у складі вузла та яку роль виконує кожний із них?
3. Перелічити рівні програмного забезпечення від мікросхеми давача до файла CSV на комп'ютері та пояснити функції кожного рівня.
4. Пояснити, який складник системи є драйвером пристрою, а який – прикладною програмою.

5. Пояснити, навіщо потрібне узгодження формату телеметричного повідомлення між прошивкою та програмою комп'ютера і до чого призводить його порушення.
6. Пояснити, чому програма мікроконтролера використовує два різні порти UART та яке призначення кожного з них.
7. Привести основні етапи відлагодження вузла у разі, коли давач опитується успішно, а прикладна програма не знаходить жодного зчитування.
8. Пояснити роботу написаних програм – прошивки мікроконтролера і прикладної програми комп'ютера.
9. Пояснити основні принципи розроблення схеми алгоритму циклічного опитування давача із заданим інтервалом.
10. Що видно на записі логічного аналізатора в межах одного інтервалу опитування і чому групи сигналів на лініях I2C та UART чергуються?
11. Як у операційних системах сімейства Linux уніфіковано доступ до інтерфейсів через файли пристроїв `/dev/i2c-*` та `/dev/tty*`?
12. Пояснити поділ на простір користувача та простір ядра і чому записування даних у файл не є розробленням драйвера периферійного пристрою.
13. Як зміниться архітектура вузла, якщо замість інтерфейсу UART використати USB-CDC?
14. Як зміниться вузол, якщо результати вимірювання потрібно передавати не на комп'ютер, а в мережу?
15. Яку роль виконує `mock`-накопичувач у ланцюжку зберігання даних і чим запис на нього відрізняється від запису на реальний носій?

ЗМІСТ ЗВІТУ

1. Мета роботи.
2. Короткі теоретичні відомості. Навести структурну схему вузла на рівні програмного забезпечення.
3. Хід роботи. Навести формат телеметричного повідомлення, фрагмент запису телеметрії, файл CSV та графік.
4. Висновки.
5. Текст програми (у додатку).
6. Демонстрація програми на комп'ютері.

ДОДАТКИ

ДОДАТОК А. ЕЛЕКТРОННІ РЕСУРСИ КУРСУ

Варіанти робіт

<https://dmytro-kushnir.github.io/#!/peripheral-devices/variants>

Тексти програмних драйверів та шаблони проєктів для виконання лабораторних робіт розміщено в електронному репозиторії курсу:

<https://github.com/dmytro-kushnir/ppid-labs>

Структура основних каталогів:

Програми хоста (лабораторні роботи № 1, 3, 5):

<https://github.com/dmytro-kushnir/ppid-labs/tree/main/host>

Містить модулі `uart_host.py`, `uart_device_emu.py`, `uart_pty_pair.py`, `usb_transaction.py`, `usb_scan.py`, `usb_gui.py`, `capstone_host.py`, `signal_gui.py`.

Програми побудови діаграм сигналів і кодування (лабораторна робота № 2):

<https://github.com/dmytro-kushnir/ppid-labs/tree/main/encoding>

Містить модулі `uart_plot.py`, `usb_nrzi.py`.

Шаблони проєктів симулятора Wokwi (лабораторні роботи № 4, 5):

<https://github.com/dmytro-kushnir/ppid-labs/tree/main/wokwi>

Містить проєкти `lab04-i2c-sensor`, `lab04-i2c-oled`, `lab05-capstone` та спільну бібліотеку драйверів у каталозі `lib`.

У звіті з кожної лабораторної роботи текст використаних програм наводять у додатку до звіту студента. У цьому навчальному посібнику повні лістинги програм не друкуються.

ДОДАТОК Б. РЕКОМЕНДОВАНА ЛІТЕРАТУРА

Базова

1. Парамуд Я. С. Периферійні пристрої, інтерфейси та драйвери : навч. посібник. Львів : Магнолія, 2022. 210 с. ISBN 978-617-574-248-8.
2. Парамуд Я. С., Миц А. М. Лабораторний практикум з дисципліни «Периферійні пристрої, інтерфейси та драйвери» : навч. посібник. Львів : Видавництво Львівської політехніки, 2021. 100 с. Режим доступу: <http://eom.lp.edu.ua/textbooks/nr-ppid.pdf>. ISBN 978-966-941-694-0.
3. Кіберфізичні системи: технології збору даних : монографія / О. Ю. Бочкарьов, В. А. Голембо, Я. С. Парамуд, В. О. Яцук ; за ред. А. О. Мельника. Львів : Магнолія 2006, 2019. 176 с.
4. USB Implementers Forum. USB Documents. Режим доступу: <https://www.usb.org/documents>.
5. NXP Semiconductors. I²C-bus specification and user manual (UM10204). Режим доступу: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>.
6. pySerial documentation. Режим доступу: <https://pyserial.readthedocs.io/>
7. Wokwi Documentation. Режим доступу: <https://docs.wokwi.com/>.
8. Веб-сайт курсу. Режим доступу: <https://dmytro-kushnir.github.io/#/peripheral-devices/>

Допоміжна

- Парамуд Я. С. Інтерфейси периферійних пристроїв ЕОМ : навч. посібник. К. : ІСДО, 1995. 76 с.
- Міюшкович Є. Г., Гребеняк А. В., Парамуд Я. С. Телекомунікаційні підсистеми кіберфізичних систем // Вісник Національного університету «Львівська політехніка». Серія: Комп'ютерні системи та мережі. 2016. № 857. С. 65–73.
- Міюшкович Є. Г., Парамуд Я. С. Особливості класифікації та застосування телекомунікаційних інтерфейсів у кіберфізичних системах // Вісник Національного університету «Львівська політехніка». Серія: Комп'ютерні системи та мережі. — 2015. — № 830. С. 106–115.
- Міюшкович Є. Г., Парамуд Я. С. Телекомунікаційні інтерфейси кіберфізичних систем: концепція застосування // Кіберфізичні системи: досягнення та виклики : матеріали І наук. семінару, 25–26 червня 2015 р., Львів. Львів : Українські технології, 2015. С. 28–37.

ЕЛЕКТРОННЕ НАВЧАЛЬНЕ ВИДАННЯ

**Кушнір Дмитро Олександрович
Парамуд Ярослав Степанович**

**ЛАБОРАТОРНИЙ ПРАКТИКУМ
з дисципліни
“ПЕРИФЕРІЙНІ ПРИСТРОЇ,
ІНТЕРФЕЙСИ ТА ДРАЙВЕРИ”**

Навчальний посібник

Видавець і виготівник: Видавництво Львівської політехніки
Свідоцтво суб'єкта видавничої справи **ДК № 4459 від 27.12.2012 р.**

вул. Ф. Колесси, 4, Львів, 79013
тел. + 380 32 2584103, факс +380 32 2584101
vlp.com.ua, ел. пошта: vmr@vlp.com.ua

П 18 **Кушнір Д.О. Парамуд Я. С.**
Лабораторний практикум з дисципліни “Периферійні пристрої, інтерфейси та драйвери” : навч. посібник / Кушнір Д.О., Я. С. Парамуд, – Львів: Видавництво Львівської політехніки, 2026. – 58 с. – Режим доступу: <http://eom.lp.edu.ua/textbooks/np-ppid.pdf>. – Заголовок з екрана
ISBN 978-966-941-694-0

Лабораторний практикум містить комплекс із методичних матеріалів до виконання восьми лабораторних робіт із дисципліни “Периферійні пристрої, інтерфейси та драйвери”.

УДК 004